

Tong Xiao

Jingbo Zhu

Natural Language Processing

Neural Networks and Large Language Models

NATURAL LANGUAGE PROCESSING LAB

NORTHEASTERN UNIVERSITY

&

NIUTRANS RESEARCH

October 6, 2026

Copyright © 2021-2026 Tong Xiao and Jingbo Zhu

NATURAL LANGUAGE PROCESSING LAB, NORTHEASTERN UNIVERSITY
&
NIUTRANS RESEARCH

<https://github.com/NiuTrans/NLPBook>

<https://niutrans.github.io/NLPBook>

Licensed under the Creative Commons Attribution-NonCommercial 4.0 Unported License (the “License”). You may not use this file except in compliance with the License. You may obtain a copy of the License at <http://creativecommons.org/licenses/by-nc/4.0>. Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an “AS IS” BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

October 6, 2026

Chapter 3

Words and Word Vectors

Words are the basic units of language [Jackendoff, 1992]. Most natural language systems used for expression and communication involve the creation, composition, and combination of words. Before exploring how words form larger linguistic units, it is worth first understanding what a word is. This involves two fundamental questions:

- What is the surface form of a word?
- What is the meaning of a word?

However, answering these questions is challenging because no simple rules can fully describe how words are formed or how their meanings are defined and induced. While various linguistic theories address these questions, NLP researchers focus more on two practical issues:

- **Tokenization:** Given a string, how to segment it into a sequence of words (also called **tokens**) such that these units can be used in downstream NLP tasks?
- **Word Representation Learning:** Given a corpus, how to represent each word in a computable form, and how can NLP models perform computation over this representation?

One goal of this chapter is to demonstrate how sentences are segmented using either linguistic or statistical methods. Specifically, we describe several approaches to tokenizing a string of characters into words or subwords using heuristic rules or statistical models learned from data. Another goal is to show how words can be represented as real-valued vectors. We present modern approaches to learning and evaluating these word vectors. The focus of this section is not on the derivation of formulas and models, but on the core concept of word vector representation, which serves as the foundation for many NLP systems. In the following chapters, we will see the generalization of this idea to modeling larger language units, such as sentences.

3.1 Tokenization

In computer science and related fields, the term *token* can be used in many different ways. Here we simply think of a token as a word in linguistics, although it can be something different (see Section 3.1.4). In NLP, tokenization or segmentation is a task related to **morphological**

Chinese	Input: 一直以来，完美的机器翻译是人类的梦想之一。 Output: 一直/以来/，/完美/的/机器翻译/是/人类/的/梦想/之一/。
Japanese	Input: 西日本や海はく晴れて、汗ばむ暑さとなる。 Output: 西日本/や/海/は/く/晴れて/、/汗ばむ/暑さ/と/なる/。
English	Input: She said, “Deep learning is not the solution to all world’s problems”. Output: She/said/,/“/Deep/learning/is/not/the/solution/to/all/world/’s/problems/”/.

Figure 3.1: Tokenization for different languages (slash = word boundary). For Chinese and Japanese where there are no delimiters between words, tokenization is often called **word segmentation**.

analysis [Aronoff and Fudeman, 2011]. While morphological analyzers or parsers are generally used to study the internal structure of words, tokenization is concerned with how sentences are broken down into words. It appears that we need to know how words are composed if we want to know how sentences are formed by words. This complexity is further compounded by linguistic diversity, which makes it difficult to establish a universal system for describing morphology. For example, analytic languages (such as Chinese) have little inflection, and rely on word order to convey meaning. By contrast, synthetic languages (such as French) may have rich inflection and the meaning of a word is highly influenced by morphology.

On the other hand, dividing sentences into smaller linguistic units is important in many NLP tasks, even though many of the world’s languages have little morphology. For example, Chinese is a morphologically simple language that has no explicit word boundaries. While it also makes sense to take characters as units in understanding the meaning of Chinese text, it is more desirable and reasonable to consider larger units in processing the text. Note that, even for languages having delimiters between words, such as English, we still have to tokenize sentences such that they are standardized when serving as the input and/or output of an NLP system.

In this section, we skip the discussion on what exactly a word is in morphology and syntax, but simply view tokenization as a task of adding word or token boundaries to a given string (see Figure 3.1). We will show that a sentence can be broken down into words or tokens in either a heuristic or statistical manner. Note that this process is designed to produce some units that can ease the processing of languages in NLP systems, not necessarily to make strict linguistic sense.

3.1.1 Tokenization via Rules and Heuristics

A common and simple approach to tokenization is to identify every word in a sentence by applying a set of pre-defined rules. In general, these rules are linguistically motivated and reflect our prior knowledge of what the form of a word should be. For example, consider the English example in Figure 3.1. We can define the following rules for tokenizing the sentence:

- Words do not contain spaces. Therefore, we can split the sentence into “word candidates”

using spaces.

- Every word candidate that is made up of English letters only (i.e., *a-z* and *A-Z*) is a word.
- Every punctuation mark (e.g., quote, comma, period) should be isolated to form a word.
- 's is treated as a distinct token, indicating a possessive noun.

This is perhaps the minimal rule set we can use in English tokenization. Surely, more rules can be added to cover more linguistic phenomena, e.g., words with dashes, words containing non-English letters, and so on. However, there are no standards to define such a set of rules. In practice, and particularly in NLP applications, we want a minimal set of rules to deal with most problems, and the tokenization is usually implemented by a number of **regular expressions**. Here we will not discuss these rules and regular expressions in detail, but refer the reader to a few textbooks for further reading [Lawson, 2003; Friedl, 2006; Jurafsky and Martin, 2008]¹.

Also, it is common to normalize the text before tokenization so that the input to the tokenizer is canonical. For example, for English and other alphabetic languages, **normalization** or **canonicalization** refers to a process of lowercasing words, normalizing character representation (e.g., Unicode characters), and so on. In addition, we can map different forms of a word to the same form for further generalization of the tokenization. A simple way to do this is to conflate all inflected forms of a word into its base form. In linguistics, the base form of a word is called **lemma**, and the process of mapping words to lemmas is called **lemmatization**. Here are some examples of lemmatization.

learn → *learn*
learning → *learn*
learns → *learn*
best → *good*

There are words that correspond to two or more different lemmas (often with different part-of-speech). In this case, we should select the correct lemma according to the context. In other words, lemmatization is context-dependent.

Closely related to lemmatization is **stemming**, which represents a word as its **stem**. Like lemmas, a stem is some base form of a word. However, unlike lemmas, a stem is not necessarily a valid word, although there are many words whose lemmas and stems are identical. Another difference from lemmatization is that stemming is performed on individual words, without the need for context for disambiguation. So, stemming is context-independent. There are several efficient algorithms for stemming. A popular one is **suffix stripping** [Porter, 1980]. It simply removes the suffixes *ing*, *ed*, *ion*, etc., as shown below:

¹Tokenization scripts can be found in many open-source projects, such as Moses [Koehn et al., 2007] (<https://github.com/moses-smt/mosesdecoder/blob/master/scripts/tokenizer/tokenizer.perl>) and the tokenizers in SacreBLEU (<https://github.com/mjpost/sacrebleu/tree/master/sacrebleu/tokenizers>).

Original	She said, “Deep learning is not the solution to all world’s problems”.
Normalization	she said, “deep learning is not the solution to all world’s problems”.
Tokenization	she/said/,/“/deep/learning/is/not/the/solution/to/all/world/’s/problems/”/.
Lemmatization	she/say/,/“/deep/learning/be/not/the/solution/to/all/world/’s/problem/”/.
Stemming	she/said/,/“/deep/learn/is/not/the/solut/to/all/world/’s/problem/”/.

Figure 3.2: Normalization, lemmatization, and stemming of an English sentence. In normalization, the whole sentence is lowercased. In lemmatization, every word is lemmatized and rewritten as its lemma. In stemming, the suffixes of some words are removed.

remove → *remov*
removing → *remov*
removal → *remov*
best → *best*

For more examples, Figure 3.2 shows normalization, lemmatization, and stemming results for an English sentence.

It is worth noting that the above methods are typically implemented using regular expressions, dictionary lookups, and additional heuristics. While this brief overview might suggest that tokenization is straightforward, much more work is needed to make it practical. In particular, when dealing with languages with a non-alphabetic writing system, or languages without explicit spacing between words, tokenization becomes a complex problem, and in that case, using simple rules would not be a good strategy. In the following subsections, we will reframe tokenization as a machine learning problem where the way to tokenize or segment sentences is learned from data. These methods are language-independent and can be applied to a wide range of tokenization or segmentation-like problems.

3.1.2 Tokenization as Language Modeling

Now let us move to statistical modeling of the tokenization problem. For ease of discussion, this subsection focuses on writing systems without explicit word boundaries, but these methods can generalize to languages with delimiters. Let $\mathbf{x} = x_1 \cdots x_l$ be a string of characters, and $\mathbf{y} = y_1 \cdots y_m$ be a sequence of words or tokens. We would say that $\mathbf{y}$ is a tokenization result of $\mathbf{x}$ if $\mathbf{y}$ defines a segmentation on $\mathbf{x}$. Consider the following Chinese sentence:

$\mathbf{x}$ = 机器翻译是人类的梦想之一。

We can define a segmentation on the sentence, for example²,

$$\begin{aligned} \mathbf{y} &= \text{机器翻译/是/人类/的/梦想/之一/。} \\ &= \left[\text{“机器翻译” “是” “人类” “的” “梦想” “之一” “。”} \right] \end{aligned} \quad (3.1)$$

In this way, tokenization can be framed as a problem of mapping $\mathbf{x}$ to $\mathbf{y}$. Given an input string, the output is the most likely segmentation:

$$\begin{aligned} \hat{\mathbf{y}} &= \arg \max_{\mathbf{y}} \Pr(\mathbf{y}|\mathbf{x}) \\ &= \arg \max_{\mathbf{y}} \log \Pr(\mathbf{y}|\mathbf{x}). \end{aligned} \quad (3.2)$$

Eq. (3.2) describes a prediction model we have been referencing several times in this book. However, the problem we are dealing with is easier because any valid segmentation $\mathbf{y}$ deterministically yields the string $\mathbf{x}$. Thus, finding the most likely segmentation simplifies to maximizing the prior probability $\Pr(\mathbf{y})$:

$$\begin{aligned} \hat{\mathbf{y}} &= \arg \max_{\mathbf{y}} \log \Pr(\mathbf{y}) \\ &= \arg \max_{\mathbf{y}} \log \Pr(y_1, \dots, y_m). \end{aligned} \quad (3.3)$$

It is easy to check that Eq. (3.3) in fact describes a language modeling problem. There are a few different ways to estimate the joint probability $\Pr(y_1, \dots, y_m)$. A simple method is to rewrite $\log \Pr(y_1, \dots, y_m)$ into a sum of log-scale conditional probabilities:

$$\begin{aligned} \log \Pr(y_1, \dots, y_m) &= \log \Pr(y_1) + \log \Pr(y_2|y_1) \\ &\quad + \dots + \log \Pr(y_m|y_1, \dots, y_{m-1}). \end{aligned} \quad (3.4)$$

Each conditional probability $\Pr(y_i|y_1, \dots, y_{i-1})$ can be approximated by

$$\Pr(y_i|y_1, \dots, y_{i-1}) \approx \Pr(y_i|y_{i-n+1}, \dots, y_{i-1}). \quad (3.5)$$

that is, the generation of y_i only depends on the $n - 1$ previous context words. To compute $\Pr(y_i|y_{i-n+1}, \dots, y_{i-1})$, we can either use the relative frequency methods or neural networks (see Chapter 2).

Now we can think of tokenization as a supervised learning problem. The process is outlined here:

- Prepare some sentences that are correctly segmented.
- Learn a language model $\Pr(\mathbf{y})$ on these labeled sentences.
- For a new sentence, find the “best” tokenization $\hat{\mathbf{y}}$ that maximizes $\Pr(\hat{\mathbf{y}})$, as in Eq. (3.3).

²Following the notation used previously, we use both $\mathbf{y} = y_1 \dots y_m$ and $\mathbf{y} = [y_1 \dots y_m]$ to denote a sequence of variables.

While this procedure follows a standard pipeline of supervised learning, there are several practical issues we have to iron out. First, the language model requires a predefined vocabulary to compute the probabilities for each token y_i , but new words are always around. To handle them, one way is to segment an unknown substring into characters, that is, we treat characters as words if the substring yielding these characters is not contained in the vocabulary. An alternative is to take into account all substrings that are not covered by the vocabulary, and replace them with the $\langle \text{unk} \rangle$ tag. The $\langle \text{unk} \rangle$ trick is widely adopted in state-of-the-art language models and is usually helpful.

Second, the language model described above has a bias towards short sequences because $\Pr(y_1, \dots, y_m)$ would be large if m is a small number. A general way to mitigate this bias is to introduce a length reward (or length bonus) to the model, for example,

$$\hat{\mathbf{y}} = \arg \max_{\mathbf{y}} \log \Pr(\mathbf{y}) + \lambda \cdot m, \quad (3.6)$$

or

$$\hat{\mathbf{y}} = \arg \max_{\mathbf{y}} \frac{\log \Pr(\mathbf{y})}{m^\lambda}. \quad (3.7)$$

where $\lambda \cdot m$ and m^λ reward long sequences and $\lambda > 0$ is a hyperparameter controlling how much we rely on the reward in assessing the goodness of $\mathbf{y}$. Interestingly, it is found that the length bias is not a big problem with tokenization in practice because the variance in length is small for those “good” tokenization results. For example, using a unigram language model (i.e., $n = 1$) without any length reward works well in many real-world applications. We will see a few examples in Section 3.1.4.

Third, performing $\arg \max$ is difficult because there are exponentially many tokenization candidates. However, the use of language models here enables efficient search algorithms. Consider, for example, applying a unigram language model to tokenization. For the input string $\mathbf{x}$, we keep, at each position j of $\mathbf{x}$, a state that describes the probability of the best tokenization on $x_1 \dots x_j$ (denoted as $p(j)$) as well as the last word of this tokenization. At position $j + 1$, we create a new state and compute the probability of the best tokenization on $x_1 \dots x_{j+1}$ by

$$\begin{aligned} p(j+1) &= \max_{1 \leq i \leq j} p(i) \cdot \Pr(x_{i+1} \dots x_{j+1}) \\ &= \max_{1 \leq i \leq j} p(i) \cdot \Pr(w_{[i+1, j+1]}), \end{aligned} \quad (3.8)$$

where $\Pr(w_{[i+1, j]})$ is the probability of the word spanning $x_{i+1} \dots x_j$. On the algorithmic side, Eq. (3.8) describes a dynamic programming method that has a time complexity of $O(l^2)$ for an input of length l . For the final output, we can trace back from the final state and dump the word sequence along the path of the optimal tokenization.

Note that the methods here are generic and can be applied to tokenization for other languages. For example, when applying it to English, we only need a slight update on the

format of the input: the input is not a character sequence but a sequence of the smallest possible pieces separated out by punctuation and spaces. For example, for the sentence *Is this Tom's laptop?*, we have

$$\mathbf{x} = [\text{Is this Tom ' s laptop ?}]. \quad (3.9)$$

Then, the tokenization process can proceed as in Eqs. (3.2–3.7).

3.1.3 Tokenization as Sequence Labeling

One common approach to segmenting characters into words is to assign labels from a predefined set to each character in a sequence, a process known as sequence labeling. Although we focus here on the problem of word segmentation, this framework is also applicable to various other NLP tasks, such as part-of-speech tagging and named entity recognition. Since the idea of sequence labeling has been discussed in Chapter 1, we present here how it is adapted to the tokenization task.

The label sets used in tokenization are generally simple. One example is the “IB” set. The “I” label indicates a linguistic piece inside a word, and the “B” label indicates the beginning of a word. The label set can be enriched by adding the “E” label (i.e., the ending of a word) and/or splitting the “B” label into sub-labels (e.g., B_1 and B_2 indicate the first and the second linguistic pieces of a word) [Zhao et al., 2006]. Given an input sequence $\mathbf{x}$ and a tokenization result $\mathbf{y}$, transforming $\mathbf{y}$ to a label sequence is fairly simple. Consider again the example used in the previous subsection. We can label the sequence in different formats:

$\mathbf{x}$:	机	器	翻	译	是	人	类	的	梦	想	之	一	。
$\mathbf{y}$:	机	器	翻	译	是	人	类	的	梦	想	之	一	。
{I, B}	B	I	I	I	B	B	I	B	B	I	B	I	B
{I, B, E}	B	I	I	E	B	B	E	B	B	E	B	E	B
{I, B_1 , B_2 , E}	B_1	B_2	I	E	B_1	B_1	B_2	B	B_1	B_2	B_1	B_2	B_1

Since the label sequence can be treated as another form of the tokenization, we can restate the problem as finding the best label sequence given an input:

$$\hat{\mathbf{c}} = \arg \max_{\mathbf{c}} \log \Pr(\mathbf{c}|\mathbf{x}), \quad (3.10)$$

where $\mathbf{c} = c_1 \cdots c_l$ is a label sequence of length l , and $\hat{\mathbf{c}}$ is the label sequence with the highest probability. Many methods have been proposed to model $\Pr(\mathbf{c}|\mathbf{x})$. A classic way is given by rewriting $\Pr(\mathbf{c}|\mathbf{x})$ using Bayes' rule:

$$\begin{aligned} \hat{\mathbf{c}} &= \arg \max_{\mathbf{c}} \log \frac{\Pr(\mathbf{x}|\mathbf{c})\Pr(\mathbf{c})}{\Pr(\mathbf{x})} \\ &= \arg \max_{\mathbf{c}} \log \Pr(\mathbf{x}|\mathbf{c}) + \log \Pr(\mathbf{c}). \end{aligned} \quad (3.11)$$

Since $\Pr(\mathbf{x})$ is a constant across all candidate sequences $\mathbf{c}$, it can be disregarded during the search process. $\Pr(\mathbf{x}|\mathbf{c})$ is the probability of generating the input $\mathbf{x}$ (i.e., observations) given the label sequence $\mathbf{c}$ (i.e., latent variables), and $\Pr(\mathbf{c})$ is a language model defined on the label sequence. Simplifications are in general required for a tractable model. For example, we can make a **Markov assumption** that the choice of c_i is dependent only on the choice of c_{i-1} . This leads to the **hidden Markov model (HMM)** which is widely used in generative modeling for NLP problems.

An alternative method is discriminative modeling. A common idea is to treat sequence labeling as a series of independent classification problems. For example, we can develop a local classifier that conditions the prediction of c_i on a set of features around position i . In more sophisticated models, such as **conditional random fields (CRFs)**, the context of the entire sequence can be used in the prediction. A detailed discussion of these sequence labeling models is beyond the scope of this section. The reader is referred to [Kupiec, 1992; McCallum et al., 2000; Lafferty et al., 2001] for thorough discussions of how these models are developed and applied. In addition, for a comparison of generative modeling and discriminative modeling, we refer the reader to Chapter 1.

3.1.4 Learning Subwords

Words are widely regarded as the basic units of language. However, they are not the smallest linguistic elements. Rather, they can be decomposed into smaller meaningful units, such as morphemes. This hierarchical structure, where morphemes form words, and words form phrases and sentences, highlights the central role of words in syntax. Nevertheless, in NLP, treating sentences strictly as word sequences is often suboptimal. A primary issue is data sparsity caused by rare words, which hinders effective model training. For instance, the word *uncopyrightable* rarely appears in corpora. An NLP system might treat it as an out-of-vocabulary token, even though its meaning can be derived by decomposing it into *un*, *copy*, *right*, and *able*. Another issue is that linguistics-based tokenization standards constrain the ability of models to learn optimal segmentation units automatically in a machine learning sense. Therefore, it is often more effective to identify “subword” units that, while not strictly linguistic, are better suited for machine learning systems.

1. Byte Pair Encoding

Byte Pair Encoding (BPE) is one of the most successful methods for learning **subword** units from a set of word sequences [Sennrich et al., 2016]. While the BPE approach stems from data compression [Gage, 1994], it is more often used in NLP as a solution to the open-vocabulary problem. The basic idea of BPE is that we repeatedly replace the most frequent pair of bytes in the data to form a new byte. As a result, common bytes are often involved in merging substrings of bytes, and rare bytes are often isolated and considered unique units. The outcome of BPE is a byte vocabulary that can be used to encode new data.

In NLP, a byte can roughly correspond to a character. Each entry of the vocabulary is a character sequence, called a symbol or subword. BPE begins with splitting a given text into a sequence of characters, for example, we can insert a space after each occurrence of

an English letter or a punctuation mark. However, this typically yields an excessively long sequence. While BPE itself has no restrictions on input length, a more common way is to prevent cross-word symbols for efficiency considerations. Thus, we can represent the text as a list of space-separated words, each being associated with the frequency of the word. For example, consider a word list:

```
f l o w # : 2
b l o w # : 2
f l a t # : 1
f l a g # : 4
```

where # is a special symbol indicating the end of a word³. From this word list, we can collect an initial vocabulary:

```
f : 7      b : 2
l : 9      a : 5
o : 4      t : 1
w : 4      g : 4
# : 9
```

Then, we count the occurrences of each symbol bigram:

```
f l : 7      a g : 4
l a : 5      g # : 4
l o : 4      b l : 2
o w : 4      a t : 1
w # : 4      t # : 1
```

We merge the most frequent symbol bigram “f l” into a new symbol “fl” and replace in the word list each occurrence of “f l” with “fl”:

```
fl o w # : 2
b l o w # : 2
fl a t # : 1
fl a g # : 4
```

³Instead of treating # as a separate symbol, it can be appended to the final character of each word

```
f l o w# : 2
b l o w# : 2
f l a t# : 1
f l a g# : 4
```

where “w#”, “t#”, and “g#” represent characters that occur at the end of a word.

Accordingly, the symbol “fl” is added to the vocabulary:

f : 7	b : 2
l : 9	a : 5
o : 4	t : 1
w : 4	g : 4
# : 9	fl : 7

This process is then iterated. This time, we merge the symbol bigram “fl a” and create a new symbol “fla”. As such, we have a new word list:

fl o w # : 2
b l o w # : 2
fla t # : 1
fla g # : 4

and a new vocabulary:

f : 7	b : 2	fla : 5
l : 9	a : 5	
o : 4	t : 1	
w : 4	g : 4	
# : 9	fl : 7	

This merging process can be repeated for a predefined number of iterations. The more times we perform the merge process, the larger the vocabulary is. The entries of the final vocabulary are reordered by symbol frequencies. For example, if we set the number of merge operations to 6, we will have a vocabulary, as follows:

l : 9	fla : 5	ow# : 4
# : 9	o : 4	flag : 4
f : 7	w : 4	flag# : 4
fl : 7	g : 4	b : 2
a : 5	ow : 4	t : 1

It corresponds to the word list:

fl ow# : 2
b l ow# : 2
fla t # : 1
flag# : 4

Having obtained a vocabulary like above, we can apply it to tokenize new words. The subword tokenization follows the same procedure of merging symbol bigrams as that used in building the vocabulary. Given a BPE vocabulary, we first segment the input text into character symbols. Then, we examine each symbol bigram in the sequence, and merge the one that has the highest frequency in the vocabulary. We repeat this operation until there are no further merges. Consider, for example, the following text:

tow a flag

It is first transformed into a character sequence:

t o w # a # f l a g #

By using the BPE vocabulary we have obtained, we can perform BPE merge operations on this sequence, as follows

	t o w # a # f l a g #
f l $\Rightarrow$ fl	t o w # a # fl a g #
fl a $\Rightarrow$ fla	t o w # a # fla g #
o w $\Rightarrow$ ow	t ow # a # fl a g #
ow # $\Rightarrow$ ow#	t ow# a # fl a g #
...	...
—————	t ow# a # flag#

This subword sequence can serve as input and/or output for a downstream NLP task, such as machine translation. Sometimes, we want to map subwords back to words. This is simple: we keep the space after each occurrence of the # symbol, and remove all other spaces and #. Also note that the BPE method we describe here requires word-segmented inputs, that is, we need a pre-tokenizer to roughly tokenize the input sequence into some units. This can be done by using the methods presented in Sections 3.1.1- 3.1.3.

2. WordPiece

WordPiece is very similar to BPE: it first divides the input text into the smallest symbols and then progressively merges consecutive symbols to form larger symbols [Schuster and Nakajima, 2012]. The difference between them is only in the way of selecting which symbol bigram to merge. At each step, BPE merges the symbol bigram with the highest frequency. Let (x_i, x_{i+1}) be a bigram in the sequence $\mathbf{x}$. The merge rule of BPE can be described as

$$(x_{\hat{i}}, x_{\hat{i}+1}) = \arg \max_{i \in [1, |\mathbf{x}| - 1]} \text{count}(x_i, x_{i+1}), \quad (3.12)$$

where the function $\text{count}(x_i, x_{i+1})$ returns the frequency of (x_i, x_{i+1}) in the corpus, and $(x_{\hat{i}}, x_{\hat{i}+1})$ is the bigram with the highest frequency.

The WordPiece method, instead, adopts a maximum likelihood criterion for bigram selection. Specifically, it merges the bigram that yields the greatest increase in the likelihood of the training data. This can be formalized as:

$$\begin{aligned} (x_i, x_{i+1}) &= \arg \max_{i \in [1, |\mathbf{x}| - 1]} \log \frac{\Pr(x_i, x_{i+1})}{\Pr(x_i) \Pr(x_{i+1})} \\ &= \arg \max_{i \in [1, |\mathbf{x}| - 1]} [\log \Pr(x_i, x_{i+1}) - \log (\Pr(x_i) \Pr(x_{i+1}))]. \end{aligned} \quad (3.13)$$

The term $\log \Pr(x_i, x_{i+1}) - \log (\Pr(x_i) \Pr(x_{i+1}))$ represents the increase in the log-likelihood of the corpus upon replacing consecutive symbols (x_i, x_{i+1}) with the merged symbol $x_i x_{i+1}$ ⁴. Thus, applying this merge rule yields a sequence of operations that incrementally increases the overall data likelihood. The outcome of this process is a codebook (i.e., a vocabulary) by which we can define the most likely code sequence for the given text.

3. SentencePiece

Both the BPE and WordPiece methods require that the input text be pre-tokenized into words. This dependency adds complexity to the tokenization pipeline. As an alternative, SentencePiece offers a more general framework that processes raw text directly by treating all characters, including spaces, as basic atomic units [Kudo and Richardson, 2018]. While SentencePiece can be paired with various subword algorithms, it is often associated with unigram language models (or simply the unigram method), which adopts a fundamentally different vocabulary construction strategy: rather than building the vocabulary bottom-up, it begins with a big vocabulary and progressively prunes it to maximize the overall data likelihood [Kudo, 2018]⁵.

The unigram method frames subword segmentation as a unigram language modeling problem, resembling the general form of Eqs. (3.3–3.4). Let $\mathbf{x}$ be a sequence of characters and $\mathbf{y}$ be a sequence of symbols or subwords that concatenates to form $\mathbf{x}$. The probability of $\mathbf{y}$ is given by

$$\Pr(\mathbf{y}) = \prod_{i=1}^{|\mathbf{y}|} \Pr(y_i). \quad (3.14)$$

Then, we can write the likelihood of $\mathbf{x}$ in terms of the joint probability of $\mathbf{x}$ and $\mathbf{y}$:

$$\Pr(\mathbf{x}) = \sum_{\mathbf{y} \in Y(\mathbf{x})} \Pr(\mathbf{x}, \mathbf{y}), \quad (3.15)$$

where the sum is over all possible tokenization results $Y(\mathbf{x})$. Since $\mathbf{x}$ is deterministically derived from $\mathbf{y}$, the joint probability simplifies to $\Pr(\mathbf{x}, \mathbf{y}) = \Pr(\mathbf{y})$. Thus, we can rewrite

⁴In statistics, $\frac{\Pr(a,b)}{\Pr(a)\Pr(b)}$ is called the **pointwise mutual information** of variables a and b . See more details in Section 3.3.1. This concept is also known as **information gain** and can be interpreted by using the Kullback-Leibler divergence or other measures in information theory (see Chapter 1).

⁵The term *vocabulary size* typically refers to the number of entries in the vocabulary. In some computational contexts, however, it may denote the total number of bytes required to store the vocabulary.

Eq. (3.15) as:

$$\begin{aligned}\Pr(\mathbf{x}) &= \sum_{\mathbf{y} \in Y(\mathbf{x})} \Pr(\mathbf{y}) \\ &= \sum_{\mathbf{y} \in Y(\mathbf{x})} \prod_{i=1}^{|\mathbf{y}|} \Pr(y_i).\end{aligned}\tag{3.16}$$

Taking this equation, the log-likelihood of a set of strings X is given by

$$\begin{aligned}\Pr(X) &= \log \prod_{\mathbf{x} \in X} \sum_{\mathbf{y} \in Y(\mathbf{x})} \prod_{i=1}^{|\mathbf{y}|} \Pr(y_i) \\ &= \sum_{\mathbf{x} \in X} \log \left(\sum_{\mathbf{y} \in Y(\mathbf{x})} \prod_{i=1}^{|\mathbf{y}|} \Pr(y_i) \right).\end{aligned}\tag{3.17}$$

If we consider $-\Pr(X)$ as a loss function, then the task here can be stated as finding the best estimate for each unigram probability $\Pr(y)$ so as to make $\Pr(X)$ as large as possible. At first glance this optimization problem looks complicated. Fortunately, there are several powerful tools to solve it. A popular method is to use **the Expectation-Maximization (EM) algorithm** [Dempster et al., 1977], which is commonly used when one tries to find a statistical model that maximizes the likelihood of the data. Note that the EM-based solution to Eq. (3.17) is similar to those for other NLP problems, such as statistical machine translation, and has been well discussed in those contexts. So we refer the reader to [Brown et al., 1993] for details about these methods. In this chapter we use EM as an off-the-shelf tool to estimate $\Pr(y)$ given Eq. (3.17).⁶

SentencePiece is essentially a “pruning” method that removes low probability entries from the vocabulary. It starts with a big initial vocabulary V . For example, we can create the initial

⁶In EM, we can view X as an observation, and $\Pr(X|\theta)$ as a statistical model that describes how likely the observation occurs. Here θ is the model parameters that we intend to determine. EM is based on an objective of maximum likelihood estimation, that is

$$\hat{\theta} = \arg \max_{\theta} \Pr(X|\theta).\tag{3.18}$$

For the model here, we can view $\{\Pr(y)\}$ as model parameters. We skip the derivation details about the EM estimate of $\Pr(y)$ but directly present the result. The EM algorithm involves two steps.

- **The Expectation Step** (or the E-step): Given the current estimate of $\Pr(y)$ (say, $\Pr_t(y)$), we compute the posterior $\Pr_t(\mathbf{y})$ for each $\mathbf{y}$ according to Eq. (3.14). Then, we compute the **fractional count** of each subword y in the vocabulary V , as follows

$$\text{fcount}(y) = \sum_{\mathbf{x} \in X} \sum_{\mathbf{y} \in Y(\mathbf{x})} \left(\Pr_t(\mathbf{y}) \sum_{i=1}^{|\mathbf{y}|} \delta(y, y_i) \right),\tag{3.19}$$

where $\delta(y, y_i)$ returns 1 if $y = y_i$, and 0 otherwise. $\sum_{i=1}^{|\mathbf{y}|} \delta(y, y_i)$ counts the number of times y occurs in the subword sequence $\mathbf{y}$.

- **The Maximization Step** (or the M-step): Given the fractional counts obtained in the E-step, we re-estimate

subword sequence	unigram probabilities ([subword]:probability)
t/ow/_/a/_flag	[t]:0.030 [ow_]:0.002 [a]:0.041 [_flag]:0.001
t/ow/_/a_/f/lag	[t]:0.030 [ow]:0.005 [_]:0.113 [a_]:0.093 [f]:0.041 [lag]:0.002
t/ow/_a_/fla/g	[t]:0.030 [ow]:0.005 [_a_]:0.084 [fla]:0.003 [g]:0.027
tow/_/a_/f/lag	[tow]:0.001 [_]:0.113 [a_]:0.093 [f]:0.041 [lag]:0.002
t/ow/_a_/flag	[t]:0.030 [ow_]:0.002 [a_]:0.093 [flag]:0.001

Figure 3.3: Different tokenization results for *tow a flag*. Every subword is assigned a probability that is estimated through a unigram language model. Every whitespace is replaced with “_” for a clear presentation.

vocabulary by enumerating all strings with a length constraint. Typically, cross-word strings are excluded to reduce the vocabulary size. Then, we run the following steps:

- Estimate the probability for each entry y of V by optimizing Eq. (3.17).
- Compute the loss for each entry y of V via the remove-one strategy, that is, the loss is the reduction in the likelihood (see Eq. (3.17)) when y is removed from the vocabulary.
- Remove a certain percentage of entries of V with large losses. For example, we keep 80% of the entries, and discard the rest.

The outcome of this process is a new vocabulary as well as the probability assigned to each subword. We can repeat this process a number of times until the vocabulary size is reduced to a desirable level.

SentencePiece differs from BPE and WordPiece in that it considers all possible subword sequences for a given string (see the sum $\sum_{y \in Y(\mathbf{x})}$ in Eq. (3.15)). From the machine learning point of view, this can be seen as a way of regularization, that is, we can reduce the risk of overestimating the parameters corresponding to the single-best subword sequence that may have errors. An alternative way is to only consider some of the subword sequences in $Y(\mathbf{x})$ for the sake of efficiency. For example, we can sample k subword sequences according to $\Pr(y)$ to form the candidate set $Y(\mathbf{x})$.

Note that the SentencePiece method does not depend on word-separated input sequences. While the BPE and WordPiece methods can also deal with raw text if updated, the SentencePiece method explicitly takes the space and other delimiters as parts of the subwords. See Figure 3.3 for a few tokenization results for *tow a flag*.

Given a learned vocabulary and the corresponding unigram probabilities, we can apply them to deal with a new text. This is in fact a search problem: we find the most likely subword

the unigram probabilities by the equation:

$$\Pr_{t+1}(y) = \frac{\text{fcount}(y)}{\sum_{y' \in V} \text{fcount}(y')}. \quad (3.20)$$

The two steps are iterated for a number of rounds until the parameters converge to some values.

sequence in terms of the unigram probability. As language modeling is a well-studied topic in NLP, many search algorithms are directly applicable to the case here. For example, the methods presented in Section 3.1.2 are straightforwardly applicable here.

3.2 Vector Representation for Words

Words inherently carry meaning⁷. In the broadest sense, the meaning of a word is derived from how it is interpreted. This is something behind the surface form of a word but can be understood by language speakers. For example, consider the following lines of text from a poem [Knight, 2018]:

*There was a little sparrow
Who sat on a wheelbarrow,
And tweeted to all her friends around.
A cat with open jaws
And very pointed claws,
Spied her as he raced along the ground.*

These words are not merely arbitrary strings of characters; they evoke specific concepts. For example, “little” means *small in size*, “sparrow” means *a kind of bird*, and “friends” means *people who you like and trust*. From an NLP perspective, however, word meaning (or **word sense**) cannot merely remain a cognitive abstraction; it must be formalized into a computable, machine-readable format.

3.2.1 One-hot Representation

A natural way to represent word meanings is to use language to describe them. For example, we can look up the aforementioned words in a dictionary to find their ids and definitions⁸:

⁷While we have so far discussed several linguistic elements used in NLP, such as subwords, we still use *words* as the basic units in our discussion here. The methods we will present in the remaining part of this chapter can generally be applied to other linguistic units used in NLP systems, including characters, subwords, and so on.

⁸All these words and their meanings are found in <https://dictionary.cambridge.org/>.

cat	511	<i>A small animal with fur, four legs, a tail, and claws, usually kept as a pet or for catching mice</i>
her	5220	<i>Used, usually as the object of a verb or preposition, to refer to a woman, girl, or female animal that has just been mentioned or is just about to be mentioned</i>
jaws	6186	<i>The mouth, including the teeth</i>
ground	6402	<i>The surface of the earth</i>
sparrow	8331	<i>A common, small, gray-brown bird</i>
wheelbarrow	9954	<i>A large, open container for moving things in with a wheel at the front and two handles at the back, used especially in the garden</i>

To represent a word, the simplest idea is to replace it with its dictionary id. In this way, each word representation is a unique number. An equivalent form to this is the **one-hot** representation. It is a vector whose dimensionality is equal to the vocabulary size. In this vector, only the element corresponding to the word index is set to 1, while all other elements are exactly 0. For example, the word *sparrow* can be represented as a one-hot vector $\mathbf{1}_{\text{sparrow}}$ based on its id (8331):

$$\begin{array}{ccccccccccc}
 [& 0 & 0 & \dots & 0 & & 1 & & 0 & \dots & 0 & 0 &] \\
 & & & & & & \uparrow & & & & & & \\
 & & & & & & \text{id} = 8331 & & & & & &
 \end{array}$$

3.2.2 Distributed Representation

However, it appears that the one-hot representation only provides the “identity” of the word but not the “description” of what the word is. An obvious problem is that every word is orthogonal to other words. Because of this orthogonality, it is impossible to mathematically quantify semantic relationships, even for words that are highly related in natural language. Here, our desire is a model in which words are described as quantifiable attributes and semantic closeness can be naturally captured. A way to do this is to enrich the representation with the word description. Consider again the word *sparrow* for example. In the dictionary, we have its meaning *a common, small, gray-brown bird*. By using the tokenization and normalization methods mentioned in Section 3.1.1, this text can be transformed into a sequence of words

$$[a \text{ common} , \text{ small} , \text{ gray} - \text{ brown} \text{ bird}]$$

Then, we vectorize this sequence using the bag-of-words model (see Chapter 1), leading to a new vector of numbers

$$\begin{array}{cccccccccccccccccccc}
 [& 0 & 0 & \dots & 1 & \dots & 1 & \dots & 1 & \dots & 1 & \dots & 1 & \dots & 1 & \dots & 1 & \dots & 0 & 0 &] \\
 \uparrow & & \uparrow & & \uparrow & & \uparrow & & \uparrow & & \uparrow & & \uparrow & & \uparrow & & \uparrow & & & & \\
 , & - & a & & bird & & brown & & common & & gray & & small & & & & & & & &
 \end{array}$$

where the value of an entry is 1 if the corresponding word is present, and 0 otherwise. This way enables the sharing of content among words. We would say that two words are similar if they have overlaps in their word vectors. Consider a new word *cuckoo*. We can find its meaning in a dictionary, e.g., *a grey bird with a two-note call that sounds similar to its name*. It is easy to know that *sparrow* and *cuckoo* are two words that share something similar because they both mark the “bird” dimension as 1 and the vector similarity between the two word vectors is greater than 0⁹.

Treating words as vectors of numbers offers a general tool to represent words in various different ways. We do not even have to explain a word vector from the viewpoint of semantics. For example, we can introduce a new dimension into the vector to mark if the word belongs to some syntactic category. In a broad sense, we can define an arbitrary function on each entry of the vector and view the function’s output as a feature describing the word. For example, a simple improvement to the above representation is to use a function counting the occurrences of a word instead of the binary-valued function marking the presence of the word. More feature functions can be found in Section 3.3.

Note that it is not necessary to constrain the feature functions to forms that make linguistic sense although linguistically motivated designs of the feature functions are usually of interest to NLP researchers. A more general form for word representation is simply a real-valued, multi-dimensional vector. It is often called the **distributed representation** of a word, or the **word embedding**. For example, the word *sparrow* can be represented as a vector

$$\begin{bmatrix} 1.9 & -7 & 3 & -1.2 & \dots & 2.01 & -2.05 \end{bmatrix}$$

From a machine learning perspective, this vector can describe some underlying attributes of a word. These attributes may not be explainable in human understanding but can be learned from data. One of the challenges in learning such a representation is that one can hardly measure the goodness of a vector. In general, it makes no sense to ask whether the distributed representation of a single word is good or not. Rather, we would like to know if the representations of a group of words are well behaved. For example, it is a common belief that similar words should have similar representations. So, the relationship between words is often thought of as some “distance” between the word representations in a vector space. This leads to a number of methods to visualize and evaluate word representations. In Section 3.6, we will give a more detailed discussion about these issues.

Typically, word representations do not work alone in NLP systems but are used as some intermediate states of a model. A standard approach in NLP, to learn distributed representations of words, is to take it as a by-product of training a “big” system. That is, the representation model works as a component of a system, and is optimized together with other components when the system is trained in some way. This inspires a promising paradigm of representation learning: the representation model is learned as a sub-model in an easy-to-train system, and can be used as a plug-in for a completely different system. In neural language modeling, for instance, we can force the model to map each input one-hot word vector into a real-valued,

⁹The similarity of two vectors can be measured by the cosine of the angle between them.

low-dimensional distributed representation. These distributed representations are fed into a neural network that predicts a probability distribution over the vocabulary. The mapping function or embedding function is trained so as to minimize the loss of the language model on some data (see Section 3.4). When applying the learned embedding function, we drop all other parts of the language model and use the function to generate the distributed representation for each word in downstream tasks. An alternative strategy is to specifically tailor the model to the word representation learning problem. Systems of this kind are typically not designed to deal with standard NLP problems, but with an emphasis on specific problems in word representation learning, such as explicitly modeling the relationship between words (see Section 3.5).

3.2.3 Compositionality and Contextuality

While our discussion is restricted to word representation learning in NLP, studying word meaning is a traditional sub-field of linguistics. In **lexical semantics**, for instance, researchers investigate how word meanings are defined and how they compose sentence-level meaning. Historically, computational word representation has not strictly adhered to linguistic semantics, focusing instead on transforming linguistic units into computational formats. However, because semantics is fundamental to language understanding, it remains crucial to incorporate semantic principles into the design of representation models. Consider the following properties:

- **Compositionality.** Compositionality is a common concept in semantics, logic and related fields. It often comes out with **the principle of compositionality**:

The meaning of a complex expression is determined by its structure and the meanings of its constituents.

– Szabó [2020]

The principle of compositionality is fundamental and exists everywhere in natural language. For instance, the meaning of the phrase *white cat* is easily derived from the individual meanings of *white* and *cat*. Similarly, compound sentences derive their overall meaning from the combination of independent clauses and conjunctions. Note that the principle of compositionality is not merely a simple rule that we use to describe how larger linguistic units are composed of smaller ones, although researchers have attempted to define it formally [Montague, 1974]. There are ongoing debates regarding how this principle should be interpreted and adequately modeled by semantic theories. Still, if we focus on NLP applications and set aside theoretical linguistic debates, compositionality remains a highly valuable property for system design and evaluation. When a problem exhibits compositionality, it implies that complex tasks can be systematically decomposed into more manageable sub-components. For word representation learning, we desire resulting vectors that exhibit compositionality to reflect the combinatorial nature of language. In Section 3.6, we will see several examples of this. For example, representations learned by neural networks often demonstrate meaningful algebraic

properties under linear operations, even though the models themselves are non-linear. However, compositionality is not an absolute rule. It frequently breaks down in cases such as collocations and idioms, where natural language exhibits non-compositionality. This explains why NLP problems are so challenging.

- **Contextuality.** Contextuality represents some sort of non-compositionality. It states that a word may have multiple meanings, and its specific interpretation is determined by the surrounding context. For example, consider the following sentences¹⁰

They sat round the dinner table, arguing about politics.

Come to the table everybody - supper's ready.

He came in with four shopping bags and dumped them on the table.

The table can help you evaluate the potential risks of investing in the Fund.

Building societies dominate the best-value tables for mortgages.

This table represents export sales.

In these example sentences, *table* is a polysemous word with two meanings:

Sense 1: *a flat surface used for putting things on.*

Sense 2: *an arrangement of items in rows, or columns, or blocks.*

In other words, *table* is an ambiguous word. This ambiguity is typically resolved by examining the adjacent words. For example, when *table* follows *dinner*, it is easy to figure out that it refers to sense 1. The ambiguity also exists when a word stems from a few different forms or lexemes (a phenomenon known as **homonymy**). For example, *bear* can be either a verb or a noun. Disambiguating a word for a given set of word senses has been studied for decades in NLP and is commonly known as **word sense disambiguation (WSD)** [Kelly and Stone, 1975]. However, the word representation problem discussed here is more challenging because we usually do not have a pre-defined set of word senses in hand. We instead want a contextual representation model that can generate a word representation dependent on its context. Thus, it is important to recognize that word meaning is not static. This makes the problem somewhat different from what we discussed at the beginning of the section, as we no longer have a lookup table for word representations, but a model that produces different representations of a word in different contexts.

In the remaining sections of this chapter, we focus on learning vector representations of words from their distributions in language use. We leave the discussion on the contextual models for learning dense word representations to Chapters 4-6.

¹⁰All these sentences are from <https://dictionary.cambridge.org/dictionary/english/table>

3.3 Count-based Models

We have framed the induction of word meanings as a problem of learning word vectors. In this section, we proceed by assuming that the meaning of a word is determined by the environment where the word is used. This is usually stated as the **distributional hypothesis** — words are semantically similar if they appear in similar contexts [Harris, 1954; Firth, 1957]. A word representation learned under this hypothesis is also called the **distributional word representation** or **distributional representation**¹¹. To ease the reading, however, we will still use the terms *word vector* and *word representation* throughout this book. Next, we introduce several methods for modeling the distribution of words in texts, and then offer some refinements.

3.3.1 Co-occurrence Matrices

In **distributional semantics**, words are represented with semantic models that consider various aspects of the context. These models differ in how the context of a word is modeled, for example, how large the context is considered, how each occurrence of a word is counted, how the dimensionality of a distribution is defined, and so on. In this section we assume, as in most models used in NLP, that word representations are learned from a collection of documents.

A document can be viewed simply as an unordered collection of words (i.e., the bag-of-words assumption). Then we can think of each occurrence of these words as an independent context indicator. In this way, the distribution of a word in its context can be described as the number of times the word co-occurs with the context words. This is achieved by constructing a co-occurrence matrix, where each cell records the co-occurrence frequency of a row item and a column item. Consider, for example, the following documents¹²:

Doc 1 *A berry is a small, pulpy, and often edible fruit.*

Doc 2 *In botanical terminology, a berry is a simple fruit with seeds and pulp produced from the ovary of a single flower.*

Doc 3 *The term "banana" is also used as the common name for the plants that produce the fruit.*

Doc 4 *Banana seeds are large and hard and spiky and liable to crack teeth.*

Doc 5 *A banana is an elongated, edible fruit - botanically a berry - produced by several kinds of large herbaceous flowering plants in the genus Musa.*

For each pair of words, we collect the total number of times they co-occur in these documents, leading to a matrix, called the **word-word co-occurrence matrix** or **term-term**

¹¹It should be noted that *distributional representation* and *distributed representation* are two different concepts. A distributional representation refers to a representation that describes the distribution of language items in language use. A related term is *non-distributional representation* which means something that is obtained from lexical databases, such as the interpretation of a word in a dictionary. On the other hand, a distributed representation refers to a vector of variables corresponding to some underlying attributes of a language item. In contrast to distributed representation, a one-hot representation just describes the word symbol.

¹²The texts are from Wikipedia.

co-occurrence matrix. Here is a subset of the matrix for the above documents.

	<i>flowering</i>	<i>fruit</i>	<i>herbaceous</i>	<i>...</i>	<i>often</i>	<i>plants</i>	<i>seeds</i>
<i>berry</i>	1	3	1	...	1	1	1
<i>terminology</i>	0	1	0	...	0	0	1
<i>common</i>	0	1	0	...	0	1	0
<i>teeth</i>	0	0	0	...	0	0	1
<i>banana</i>	1	2	1	...	0	2	1
<i>simple</i>	0	1	0	...	0	0	1
<i>and</i>	0	2	0	...	1	0	2

In the matrix, each row word is associated with a word vector of $|V|$ entries. The numbers in the entries describe how often the row word co-occurs with different context words, that is, how a given word is distributed in different “contexts”. In a geometric sense, if two words have similar distributions in co-occurring with the same group of context words, then the angle between the word vectors would be small¹³. For example, if we think of these words as vectors in a vector space, *berry* is closer to *banana* than *teeth* (see Figure 3.4). This geometric intuition is the basis of many representation models. More examples will be given in Chapters 4 and 5.

A problem with this method is that the distance between words is not taken into account although the correlation is not that strong when the context word is distant. A simple solution is to constrain context words in a window, called the **context window** or window for short [Lund and Burgess, 1996]. For example, for each word in a document, we only count the -2 and +2 words surrounding it (i.e., a window of size 5).

Note that the word vectors learned by the bag-of-words model in Section 3.2 are a special instance of the co-occurrence matrix. In that example, the entire corpus consists of a single document, and an indicator function marks presence rather than frequency. Beyond simple counts or binary indicators, the entries of a word vector can be populated using various association measures. In practice, the value of an entry of a word vector can be thought of as the degree of the correspondence between words. If two words are correlated with each other in some context, a feature function may assign a score between them in any manner. This score does not necessarily have to be a count, but can be an arbitrary real number. As such, the problem can be stated as measuring the association strength between words. It is common practice to define such a measure on the basis of correlation models. In statistics, correlation describes to what extent two variables are associated, measured by **correlation coefficients**. Common correlation coefficients include the Pearson correlation coefficient (Pearson’s r), the Spearman’s rank correlation coefficient (Spearman’s ρ), and so on¹⁴. In NLP, a widely used measure is the **pointwise mutual information (PMI)** [Church and Hanks, 1990]. Let a and b

¹³The angle between two vectors is independent of their magnitudes. If the vectors are normalized in some way (e.g., by vector norm), similar vectors mean that most entries of the two vectors have similar values.

¹⁴Some of the correlation coefficients assume certain distributions of the data. For example, the Pearson correlation coefficient is calculated based on two variables following normal distributions.

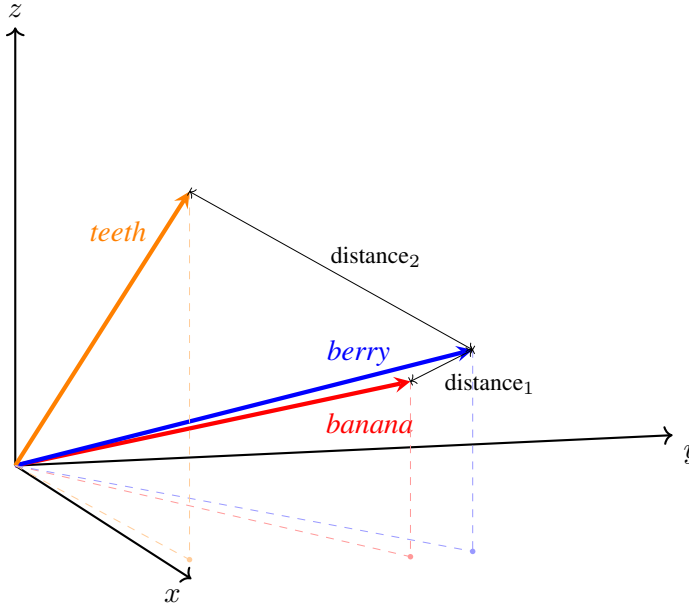


Figure 3.4: Word vectors in a vector space that is built from the word co-occurrence statistics on the English data from WMT 2012. All the vectors are normalized and represented as arrows. For visualization, we project these vectors from a high-dimensional space to a 3-dimensional space via principal component analysis. As expected, *berry* is closer to *banana* than to *teeth*.

be two words. The mathematical form of PMI is given by

$$\text{PMI}(a, b) = \log \frac{\Pr(a, b)}{\Pr(a)\Pr(b)}, \quad (3.21)$$

where $\Pr(a, b)$ is the joint probability of a and b co-occurring, and $\Pr(a)$ (or $\Pr(b)$) is the probability of a (or b) occurring. These probabilities can be simply estimated on the texts by the relative frequency method¹⁵. Given a word a and a vocabulary of context words $\{b_1, \dots, b_{|V|}\}$, the PMI-based word vector of a is written as

$$e(a) = \left[\text{PMI}(a, b_1) \quad \dots \quad \text{PMI}(a, b_{|V|}) \right]. \quad (3.22)$$

Correlation coefficients are generally used to test whether two variables are (linearly) related. So, an alternative method is to define an entry of the word vector as the outcome of a test. For example, the entry (a, b) chooses a value of 1, if the correlation coefficient between words a and b is larger than a threshold, or the correlation of words a and b is sufficiently supported by hypothesis testing.

However, modeling words as vectors of correlation scores between words somewhat limits the scope of contextual information one may use. An alternative macro-level approach is

¹⁵ A problem with PMI is that the measure becomes unstable when the words are rare. For example, if a very rare word happens to appear in a document, the PMI value of this word and any other word in this document would be unreasonably large.

Entry	Mathematical form
Binary	$\text{tf}(a, d) = \begin{cases} 1 & a \text{ occurs in } d \\ 0 & \text{otherwise} \end{cases}$
Count	$\text{tf}(a, d) = \text{count}(a; d)$
Exponential Count	$\text{tf}(a, d) = \text{count}(a; d)^\alpha$
Log-scale Count	$\text{tf}(a, d) = \log(1 + \text{count}(a; d))$
Normalized Count (or Frequency)	$\text{tf}(a, d) = \frac{\text{count}(a; d)}{\sum_{a'} \text{count}(a'; d)}$

Table 3.1: Functions of the term-frequency weighting scheme. $\text{count}(a; d)$ counts the occurrences of the word a in the document d .

to analyze how words distribute across entire documents, forming a **word-document co-occurrence matrix** or **term-document co-occurrence matrix**. For example, for the above mentioned documents, we can build a matrix

	Doc 1	Doc 2	Doc 3	Doc 4	Doc 5
<i>berry</i>	1	1	0	0	1
<i>terminology</i>	0	1	0	0	0
<i>common</i>	0	0	1	0	0
<i>teeth</i>	0	0	0	1	0
<i>banana</i>	0	0	1	1	1
<i>simple</i>	0	1	0	0	0
<i>and</i>	1	1	0	2	0

In the matrix, the value of entry (a, d) is defined to be the number of times the word a occurs in the document d , quantifying the association strength between a and d . This is commonly called the **term frequency (TF)** of a in d (denoted by $\text{tf}(a, d)$). Also, we can use a 0-1 indicator function to mark the presence of the word occurrence (see Section 3.2). See Table 3.1 for a few variations of the TF weighting function.

This structure serves a dual purpose: each row provides a vector representation of a specific word, while each column acts as a vector representation of an entire document. This word-document co-occurrence matrix is conceptually identical to the bag-of-words model introduced for text classification in Chapter 1, where word order is discarded but relative frequencies are preserved. Here we perform document vectorization via this model on a collection of documents.

3.3.2 TF-IDF

Modeling word-document associations is important for many NLP tasks. A significant refinement over raw frequency counts for building these vectors is the **term frequency-inverse**

document frequency (TF-IDF) weighting scheme. Given a set of documents D , TF-IDF assigns a score to each word-document pair (a, d) via the following equation:

$$\text{tfidf}(a, d, D) = \text{tf}(a, d) \cdot \text{idf}(a, D), \quad (3.23)$$

where

- $\text{tf}(a, d)$ is the term frequency (see Table 3.1). When $\text{tf}(a, d)$ is large, the word a is a good indicator for the document d . In contrast, when $\text{tf}(a, d)$ is small, the word-document association is not that strong.
- $\text{idf}(a, D)$ is the **inverse document frequency (IDF)**. It is developed based on the fact that common words across documents are less informative. For example, for a collection of documents on sports, it is likely to see *player* and *players* in most documents. In this case, the words *player* and *players* are less interesting in discriminating different documents or contexts. Let $\text{df}(a, D)$ be the number of documents in D containing the word a . A common form of $\text{idf}(a, D)$ is given by

$$\text{idf}(a, D) = \log \frac{|D|}{\text{df}(a, D)}. \quad (3.24)$$

This equation penalizes words that appear frequently across the corpus. Similarly, we can have a smoothed version of $\text{idf}(a, D)$

$$\text{idf}(a, D) = \log \frac{|D|}{\text{df}(a, D) + 1} + 1. \quad (3.25)$$

Using the TF-IDF weighting scheme, we can construct a word-document matrix where the value of entry (a, d) is $\text{tfidf}(a, d, D)$. As previously described, each row of this matrix serves as the vector representation of its corresponding word. Traditionally, TF-IDF and term-document matrices are extensively used for document representation. For instance, in information retrieval, both queries and documents can be represented as TF-IDF column vectors, allowing us to compute query-document relevance via vector similarity. While vector space models in information retrieval fall outside the scope of this chapter, readers are referred to standard textbooks for comprehensive coverage [Manning et al., 2008; Butcher et al., 2016].

3.3.3 Low-Dimensional Models

Co-occurrence matrices are often high dimensional. Suppose, for example, that there is a vocabulary of 20,000 unique words and a collection of 10,000,000 documents. Then, a word-document co-occurrence matrix has $20,000 \times 10,000,000 = 2 \times 10^{11}$ entries. Given the computational burden associated with such a model, it is impractical to represent a word as a 10,000,000-dimensional vector and a document as a 20,000-dimensional vector. Instead, we expect that the representation of a word (or a document) requires only a reasonably small number of features. In this subsection, we discuss standard approaches for transforming

high-dimensional co-occurrence matrices into lower-dimensional representations. Because these techniques have been extensively studied and successfully applied across various disciplines [Barber, 2012; Wright and Ma, 2022], we bypass the underlying mathematical proofs and focus instead on their practical application for deriving word and document vectors.

1. Latent Semantic Analysis

In NLP, **latent semantic analysis (LSA)** is a method of seeking the latent semantic structure behind the word-document associations [Deerwester et al., 1990; Landauer et al., 1998]¹⁶. It assumes that both words and documents can be represented as low-dimensional vectors that are distilled from the co-occurrence matrix, preserving the property of the original vector space model, e.g., the angle between vectors is small for similar words.

More specifically, LSA factorizes the co-occurrence matrix into a matrix for word representation, a matrix for document representation, and a third matrix connecting the first two matrices. Mathematically, this can be framed as a **singular value decomposition (SVD)** process [Stewart, 1993]. Let $\mathbf{M} \in \mathbb{R}^{|V| \times |D|}$ be a co-occurrence matrix over a vocabulary V and a document set D . The SVD produces a factorization of $\mathbf{M}$

$$\mathbf{M} = \mathbf{P}\mathbf{\Sigma}\mathbf{Q}^\top, \quad (3.26)$$

where $\mathbf{P} \in \mathbb{R}^{|V| \times r}$, $\mathbf{\Sigma} \in \mathbb{R}^{r \times r}$, and $\mathbf{Q}^\top \in \mathbb{R}^{r \times |D|}$. In this factorization, the matrices $\mathbf{P}$ and $\mathbf{Q}^\top$ are **semi-unitary** (or **semi-orthogonal**), meaning that the columns of both $\mathbf{P}$ and $\mathbf{Q}$ are mutually **orthonormal vectors** (orthogonal unit vectors). Consequently, the columns of $\mathbf{P}$ and $\mathbf{Q}$ form orthonormal bases for the column space and row space of $\mathbf{M}$, respectively, where r is the rank of $\mathbf{M}$. This guarantees that $\mathbf{M}$ is represented using the minimum necessary number of dimensions. The matrix $\mathbf{\Sigma}$ is diagonal:

$$\mathbf{\Sigma} = \begin{bmatrix} \sigma_1 & 0 & \cdots & 0 \\ 0 & \sigma_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_r \end{bmatrix}. \quad (3.27)$$

The diagonal entries $\{\sigma_1, \dots, \sigma_r\}$ are all non-negative real numbers, and are called the **singular values** of $\mathbf{M}$. Typically, $\{\sigma_1, \dots, \sigma_r\}$ are arranged in descending order (i.e., $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r$). This ordering standardizes the decomposition. If we write $\mathbf{P}$ as a sequence of column vectors (call them **left-singular vectors**)

$$\mathbf{P} = [\mathbf{p}_1, \dots, \mathbf{p}_r], \quad (3.28)$$

¹⁶Latent semantic analysis is also called **latent semantic indexing (LSI)**. This term is more often used in information retrieval and related fields.

and $\mathbf{Q}^\top$ as a sequence of row vectors (call them **right-singular vectors**)

$$\mathbf{Q}^\top = \begin{bmatrix} \mathbf{q}_1^\top \\ \vdots \\ \mathbf{q}_r^\top \end{bmatrix}, \quad (3.29)$$

then we can write $\mathbf{M}$ as

$$\mathbf{M} = \sum_{i=1}^r \sigma_i \mathbf{p}_i \mathbf{q}_i^\top. \quad (3.30)$$

To obtain word representations, we interpret the i -th row of $\mathbf{P}$ as an r -dimensional feature vector $\mathbf{e}_i$ for the i -th word a_i in the vocabulary. In other words, the vector representation of a_i is

$$\mathbf{e}_i = \begin{bmatrix} p_1(i) & \cdots & p_r(i) \end{bmatrix}. \quad (3.31)$$

Similarly, the vector representation of a document d_j can be written as

$$\mathbf{h}_j = \begin{bmatrix} q_1(j) & \cdots & q_r(j) \end{bmatrix}. \quad (3.32)$$

In this way, we have two separate representation models for words and documents: $\mathbf{P}$ deals with word representations and $\mathbf{Q}$ deals with document representations. Thus, we can take $\mathbf{M}$ as a product of these representation models:

$$\begin{aligned} \mathbf{M} &= \mathbf{P} \Sigma \mathbf{Q}^\top \\ &= \text{words} \begin{bmatrix} \mathbf{e}_1 \\ \vdots \\ \mathbf{e}_{|V|} \end{bmatrix} \begin{bmatrix} \sigma_1 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & \sigma_r \end{bmatrix} \text{documents} \begin{bmatrix} \mathbf{h}_1^\top & \cdots & \mathbf{h}_{|D|}^\top \end{bmatrix}. \end{aligned} \quad (3.33)$$

In practice, the rank r is usually much smaller than $|V|$ and $|D|$. Thus, we have, for each word (or each document), a new representation whose dimensionality is much smaller than the representation contained in the co-occurrence matrix. A further improvement can make use of the r^* largest singular values (i.e., $\{\sigma_1, \dots, \sigma_{r^*}\}$) and throw away the rest. As a consequence, we only keep the first r^* left-singular vectors and right-singular vectors in $\mathbf{P}$ and $\mathbf{Q}$ respectively. Here $r^* < r$ is a hyperparameter specifying the number of vectors in $\mathbf{P}$ and $\mathbf{Q}$, i.e., the number of features used to describe a word or a document. In this way, we have a new factorization of $\mathbf{M}$ as

$$\mathbf{M} \approx \sum_i^{r^*} \sigma_i \mathbf{p}_i \mathbf{q}_i^\top. \quad (3.34)$$

The right hand side of Eq. (3.34) is also known as a **low-rank approximation** of $\mathbf{M}$. By specifying r^* , it can approximate $\mathbf{M}$ with a matrix having an arbitrary rank $< r$.

There are a number of algorithms for implementing the SVD [Cline and Dhillon, 2014]. In fact, most of the modern implementations of the SVD are efficient and scalable. One can use them as off-the-shelf toolkits in NLP applications.

2. Principal Component Analysis

In data analysis, **principal component analysis (PCA)** is a widely-used technique for dimensionality reduction. Given a set of data points, PCA finds a sequence of orthogonal directions in the coordinate space so that the variance of the data points along these directions is maximized. These directions are typically represented as unit vectors, called **principal component loadings** or **principal component coefficients**. As a result, they form a new coordinate space to which we can map the given data points by an orthogonal linear transformation.

Consider a word-document co-occurrence matrix $\mathbf{M} \in \mathbb{R}^{|V| \times |D|}$, where each row is a $|D|$ -dimensional word vector or feature vector. PCA defines a linear mapping from $\mathbb{R}^{|D|}$ to $\mathbb{R}^p$, that is, we transform each $|D|$ -dimensional word vector into a p -dimensional word vector. This is given by

$$\mathbf{N} = \mathbf{MC}, \quad (3.35)$$

where $\mathbf{N} \in \mathbb{R}^{|V| \times p}$ represents the matrix of mapped word vectors over the vocabulary V , and $\mathbf{C} \in \mathbb{R}^{|D| \times p}$ is the matrix of the linear mapping. Then, we can write $\mathbf{C}$ as a sequence of column vectors

$$\mathbf{C} = \begin{bmatrix} \mathbf{c}_1 & \cdots & \mathbf{c}_p \end{bmatrix}. \quad (3.36)$$

Each column vector $\mathbf{c}_i = \begin{bmatrix} c_i(1) \\ \vdots \\ c_i(|D|) \end{bmatrix}$ is a group of principal component coefficients, indicating a linear function that combines the input features into a new feature. For example, if we view $\mathbf{M}$ as the values of a bunch of feature functions (say, column vectors $\{\mathbf{m}_1, \dots, \mathbf{m}_{|D|}\}$), we can map $\mathbf{M}$ to a new feature space in terms of $\mathbf{c}_i$:

$$\begin{aligned} \mathbf{M}\mathbf{c}_i &= \begin{bmatrix} \mathbf{m}_1 & \cdots & \mathbf{m}_{|D|} \end{bmatrix} \begin{bmatrix} c_i(1) \\ \vdots \\ c_i(|D|) \end{bmatrix} \\ &= \sum_{k=1}^{|D|} c_i(k) \mathbf{m}_k. \end{aligned} \quad (3.37)$$

The product $\mathbf{M}\mathbf{c}_i$ (which forms the i -th column of $\mathbf{N}$) is a column vector where each entry represents the new feature value for a word in V . In PCA, the vectors $\{\mathbf{c}_1, \dots, \mathbf{c}_p\}$ are generated sequentially to maximize the variance of the projected data. Thus, for each $i \in [1, p]$,

the optimal principal component coefficients are defined as:

$$\begin{aligned}\hat{\mathbf{c}}_i &= \arg \max_{\mathbf{c}_i} \text{Var}(\mathbf{M}\mathbf{c}_i) \\ &= \arg \max_{\mathbf{c}_i} \mathbf{c}_i^\top \mathbf{S} \mathbf{c}_i,\end{aligned}\tag{3.38}$$

where $\text{Var}(\mathbf{M}\mathbf{c}_i)$ is the variance of the projected data, and $\mathbf{S}$ is the $|D| \times |D|$ covariance matrix of $\mathbf{M}$. To ensure a well-defined solution, it is standard practice to constrain $\mathbf{c}_i$ to be a unit vector (i.e., $\mathbf{c}_i^\top \mathbf{c}_i = 1$). Using a Lagrange multiplier λ_i , this constrained optimization problem can be formulated by defining the Lagrangian:

$$\mathcal{L}(\mathbf{c}_i, \lambda_i) = \mathbf{c}_i^\top \mathbf{S} \mathbf{c}_i - \lambda_i (\mathbf{c}_i^\top \mathbf{c}_i - 1).\tag{3.39}$$

Solving for the stationary points of this Lagrangian reveals that $\hat{\mathbf{c}}_i$ must be an eigenvector of $\mathbf{S}$, with λ_i as its corresponding eigenvalue [Jolliffe, 2002]. Because $\mathbf{S}$ is a $|D| \times |D|$ symmetric matrix, it possesses exactly $|D|$ eigenvalues and corresponding mutually orthogonal eigenvectors. By ordering these eigenvectors according to their associated eigenvalues in descending order, we select the top p eigenvectors to form $\{\hat{\mathbf{c}}_1, \dots, \hat{\mathbf{c}}_p\}$. In other words, $\hat{\mathbf{c}}_1$ is the eigenvector associated with the largest eigenvalue, $\hat{\mathbf{c}}_2$ with the second largest, and so forth. The projection $\mathbf{M}\hat{\mathbf{c}}_i$ is termed the i -th **principal component** of $\mathbf{M}$.

A geometric way to conceptualize PCA is as a coordinate transformation in Euclidean space. For a dataset $\mathbf{M}$, each row represents the coordinates of a data point in a $|D|$ -dimensional space A . PCA seeks to represent these data points in a new, p -dimensional coordinate system B . The i -th dimension of this new system is defined by the direction of the unit vector $\mathbf{c}_i$. To find the i -th coordinate of a data point in B , we project its original coordinates onto the line spanned by $\mathbf{c}_i$. The optimal direction $\mathbf{c}_i$ is chosen to maximize the spread (variance) of these projected points. Essentially, we seek the axis along which the data points are most dispersed. Through this process, we generate a sequence of principal component coefficients by successively solving Eq. (3.38). Figure 3.5 illustrates this concept by projecting 2-dimensional data onto a 1-dimensional subspace.

In real-world applications, p is typically chosen to be much smaller than $|D|$, allowing PCA to significantly reduce the dimensionality of the word representations. Note that PCA is a very general method and is found to be useful in many disciplines. More broadly, if $\mathbf{M}$ represents observations over a set of variables, PCA can condense these observations into a smaller set of uncorrelated variables (the principal components).

3. Other Methods

Dimensionality reduction is a fundamental problem in machine learning and has been generalized in several directions. For instance, the neural word embedding models described in Sections 3.4 and 3.5 inherently learn low-dimensional, real-valued word vectors from text. Here we present some of the dimensionality reduction methods one may come across in the NLP and machine learning literature.

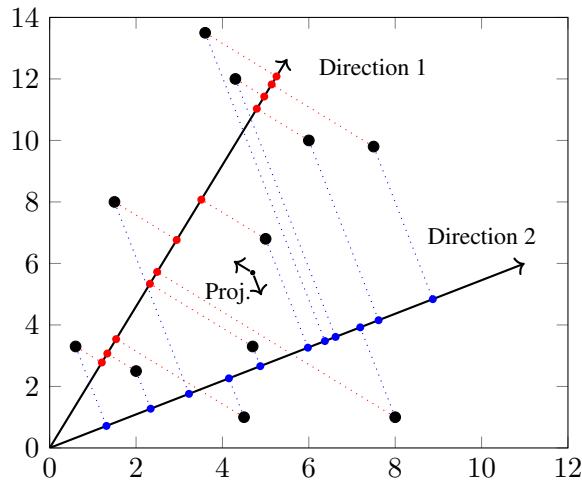


Figure 3.5: Illustration of transforming 2-dimensional data to 1-dimensional data via PCA. Consider a set of data points (represented by black circles) on a Euclidean plane. By using PCA, we find a direction (represented by an arrow) such that the variance of the projected data (represented by colored circles) in this direction is maximized. Such a direction can be represented by a unit vector of principal component coefficients. In this example, the principal component coefficients describe a 1-dimensional coordinate space. We can map the data from the 2-dimensional coordinate space to the 1-dimensional coordinate space via linear transformation. The mapped data is called the principal component of the original data points.

- **Topic models.** Technically, topic models are not ways of dimensionality reduction, but tools for describing how documents and words are generated based on distributions over topics [Blei, 2012]. For example, **latent Dirichlet allocation (LDA)** models the generation of a document by using document-topic and topic-word distributions [Blei et al., 2003]. As a by-product, we obtain a distribution over words for each topic, indicating the likelihood of a word occurring given a specific topic. If we write all these topic-word distributions as a $|V| \times K$ matrix where $|V|$ is the number of words and K is the number of topics, we obtain word representations conceptually similar to those described in previous sections. Typically, K is a relatively small number (e.g., 200). In this case, we have a low-dimensional model for representing words. Although LDA is not so popular in learning word representations in NLP applications, it offers a way to represent words as distributions over latent thematic structures.
- **Auto-encoders.** Undercomplete auto-encoders are a type of neural model that encodes features into low-dimensional codes such that the input features can be reconstructed from the codes. An advantage of auto-encoders is that they do not make assumptions on the hidden structures of the features. Thus, auto-encoders learn to transform any type of data into low-dimensional representations. For example, in Chapter 7 we will see examples of applying auto-encoders to learn sentence representations. For more details about auto-encoders, the reader can refer to Chapter 2.
- **Supervised dimensionality reduction.** Traditional dimensionality reduction methods

(such as PCA) operate in an unsupervised manner. However, when target task labels are available, it is advantageous to leverage this supervision signal. A classic example is supervised dimensionality reduction for classification. In methods like **Fisher’s linear discriminant** and **linear discriminant analysis**, high-dimensional data is mapped into a lower-dimensional space that maximizes the separation between predefined classes. This concept is generalized to multi-dimensional target spaces via Canonical Variates analysis [Barber, 2012].

- **Feature selection.** Feature selection is the process of identifying a subset of the most relevant features to represent an object, thereby explicitly reducing the representation’s dimensionality. It is a broad topic in machine learning involving many methods [Guyon and Elisseeff, 2003; Liu and Motoda, 2012]. The simplest is to frame it as a search problem: we search in the space of feature subsets so that the selected features maximize (or minimize) some objective. In general, the design of the objective depends on the task where we apply the features. This tight coupling makes feature selection challenging, as it requires balancing many factors, such as the performance measure of the target task, the search efficiency, and the representation of each feature subset. Note that feature selection is generally discussed in supervised learning, which requires labeled data to compute loss for optimization. The reader is referred to Solorio-Fernández et al. [2020]’s review paper for unsupervised feature selection methods.

In statistics, many methods can fall under the dimensionality reduction framework and are related to what we discussed in this section. For example, **factor analysis** is a method similar to PCA because they both seek a linear mapping from the input variables to a smaller number of new variables. The difference between them is that factor analysis focuses on modeling the common variance of variables, while PCA focuses on maximizing the variance of the projected data. Another example is **independent component analysis (ICA)**. Unlike PCA, the goal of ICA is to find independent components that are additively separable. More examples can be found in machine learning and statistics textbooks [McClave and Sincich, 2006; Freedman et al., 2007; Barber, 2012].

3.4 Inducing Word Embeddings from NLMs

Counting word-word or word-document occurrences is a straightforward method for representing words based on their distributional properties in a corpus. While this method is effective in many applications, it imposes a strict constraint on word representations: the entries of a word vector must be interpretable as explicit evidence of how the word distributes across different contexts. Ideally, we would like to represent words in a more generalized form — namely, as dense, real-valued vectors (commonly referred to as **word embeddings**) without enforcing predefined constraints or assumptions on the specific meaning of each vector entry.

Learning word vectors without constraints comes at a cost. Unlike the count-based methods presented in Section 3.3, we do not rely on heuristics or prior statistical knowledge to estimate the vector values. Instead, we wish to induce meaningful representations directly from data.

A primary challenge here is the absence of a ground-truth gold standard, as it is impractical to manually annotate high-dimensional, continuous representations for an entire vocabulary. Thus, we frequently frame the induction of word vectors as an auxiliary objective within a well-defined task (often referred to as a **background task**). The learned word vectors emerge as a valuable byproduct of training on this background task.

A common example is the induction of word vectors from neural language models (NLMs). Recall the NLM described in Chapter 2. Its goal is to predict the probability of a word given its preceding words [Bengio et al., 2003]. More formally, let w_i be the word we want to predict, and $\{w_{i-n+1}, \dots, w_{i-1}\}$ be the context words we have seen. First, the context words $\{w_{i-n+1}, \dots, w_{i-1}\}$ are projected into d_e -dimensional dense vectors $\{\mathbf{e}_{i-n+1}, \dots, \mathbf{e}_{i-1}\}$ via an embedding layer. Assuming $\mathbf{1}_{w_j}$ is the one-hot representation of word j (a row vector of size $|V|$), the dense word vector $\mathbf{e}_j$ is given by

$$\mathbf{e}_j = \mathbf{1}_{w_j} \mathbf{C}, \quad (3.40)$$

where $\mathbf{C} \in \mathbb{R}^{|V| \times d_e}$ is the parameter of the embedding layer. $\mathbf{C}$ is often known as the word embedding table in which the k -th row is the representation of the k -th word in V .

Then, we use a feed-forward neural network to compute the probability distribution of the word at position i . This is given by

$$\Pr(\cdot | w_{i-n+1}, \dots, w_{i-1}) = F_\theta(\mathbf{e}_{i-n+1}, \dots, \mathbf{e}_{i-1}), \quad (3.41)$$

where $F_\theta(\cdot)$ is a feed-forward neural network parameterized by θ . Typically, the embedding layer can be seen as a component of the NLM. Here we use slightly different notation to emphasize that the NLM is a function of both θ and $\mathbf{C}$

$$\Pr_{\theta, \mathbf{C}}(\cdot | w_{i-n+1}, \dots, w_{i-1}) = F_{\theta, \mathbf{C}}(w_{i-n+1}, \dots, w_{i-1}). \quad (3.42)$$

For training, we optimize both θ and $\mathbf{C}$ to minimize a loss function. A popular method is maximum likelihood training which maximizes the sum of log-likelihood over all n -grams in the data. Given a sequence of words $w_1 \dots w_m$, the objective of the training is defined to be:¹⁷

$$(\hat{\theta}, \hat{\mathbf{C}}) = \arg \max_{\theta, \mathbf{C}} \sum_{i=n}^m \log \Pr_{\theta, \mathbf{C}}(w_i | w_{i-n+1}, \dots, w_{i-1}). \quad (3.43)$$

Having obtained the optimized parameters $\hat{\theta}$ and $\hat{\mathbf{C}}$, we can apply $F_{\hat{\theta}, \hat{\mathbf{C}}}(\cdot)$ to deal with unseen n -grams. More importantly, we have some well-trained word vectors (i.e., $\hat{\mathbf{C}}$) that can be used in systems other than NLMs. This is also known as the pre-training of word vectors. In pre-training, we can define $F_{\theta, \mathbf{C}}(\cdot)$ as any system that makes use of the word vectors $\mathbf{C}$. Thus, the task of learning $\mathbf{C}$ is transformed to the task of optimizing $F_{\theta, \mathbf{C}}(\cdot)$ on the background task (see Figure 3.6 for an illustration). The main advantage of this method is that we can reuse existing NLP tasks to train the word vectors. A risk here is that the “best” word vectors found

¹⁷This can be generalized to a data set consisting of multiple sequences.

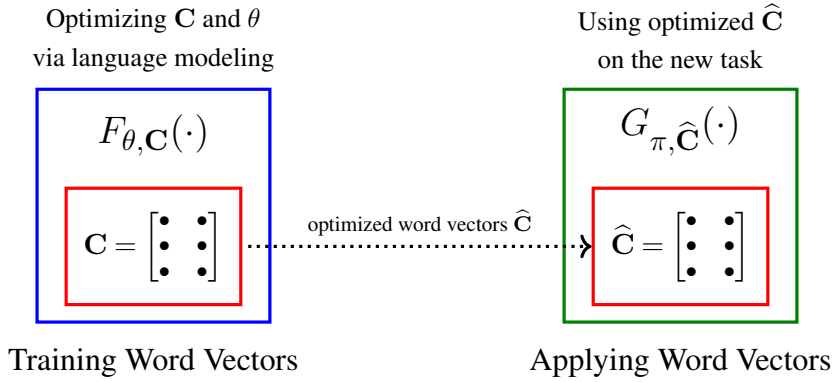


Figure 3.6: Illustration of pre-training word vectors in an NLM. The NLM can be denoted as a function $F_{\theta, \mathbf{C}}(\cdot)$ of the word embedding table (i.e., $\mathbf{C}$) and other parameters of the NLM (i.e., θ). The pre-training of $\mathbf{C}$ is essentially a process of training $F_{\theta, \mathbf{C}}(\cdot)$ on a background task. The outcome is the optimized word vectors $\hat{\mathbf{C}}$ which are then applied to a new system $G_{\pi, \hat{\mathbf{C}}}(\cdot)$ that might be different from the NLM. In the new system, $\hat{\mathbf{C}}$ is the word embedding table learned from the NLM and π represents the parameters specialized to $G(\cdot)$.

in training $F_{\theta, \mathbf{C}}(\cdot)$ might not be well suited for the system where the word vectors are in actual use. Interestingly, in many situations, word vectors that are pre-trained by NLMs are of good quality for downstream tasks, or at least provide a good starting point for further tuning of these word vectors in the target system.

3.5 Word Embedding Models

In principle, word vectors can be learned in any manner. Treating word vectors as components of existing NLP systems is one option, but this approach typically lacks task-specific considerations. Another option is to develop methods specifically tailored to the problem of learning representations. Because the training of such systems does not need to satisfy the strict constraints of standard downstream NLP tasks, inducing high-quality word vectors becomes significantly more tractable.

3.5.1 Word2Vec

Word2Vec refers to a suite of models proposed in [Mikolov et al., 2013a;c]. As with neural language models, Word2Vec models are built upon neural networks. However, rather than resorting to the heavy autoregressive generative modeling of n -grams, Word2Vec frames the learning of word vectors in a computationally efficient log-linear fashion. Consequently, the architectures of these models differ markedly from those used in traditional language modeling. There are two primary architectures in the Word2Vec family:

- **The continuous bag-of-words model (or the CBOW model).** The CBOW model is a word prediction model. It is used to predict how likely a word at position i occurs given the $-n$ and $+n$ word windows around it. The structure of the CBOW model is

similar to that of the neural language model introduced in Chapter 2 (see Figure 3.7 (a)). First, we use an embedding layer to transform the context words $w_{i-n} \cdots w_{i-1}$ and $w_{i+1} \cdots w_{i+n}$ to corresponding word vectors. This is performed by multiplying the one-hot representation of each input word w_j with the embedding table $\mathbf{C} \in \mathbb{R}^{|V| \times d_e}$, as shown in Eq. (3.40). These word vectors are then averaged to produce a single representation for the input words, giving us

$$\mathbf{h} = \frac{1}{2n} \left(\sum_{j=i-n}^{i-1} \mathbf{1}_{w_j} \mathbf{C} + \sum_{j=i+1}^{i+n} \mathbf{1}_{w_j} \mathbf{C} \right). \quad (3.44)$$

Note that the above defines a model that completely ignores the order of input words because of the use of the sum operation. This explains why the CBOW model is called *bag-of-words*. The output layer of the CBOW model is a standard Softmax layer that projects $\mathbf{h}$ to a probability distribution over the vocabulary

$$\mathbf{y} = \text{Softmax}(\mathbf{h}\mathbf{U} + \mathbf{b}), \quad (3.45)$$

where $\mathbf{U} \in \mathbb{R}^{d_e \times |V|}$ is the parameter matrix of the linear mapping and $\mathbf{b} \in \mathbb{R}^{|V|}$ is the bias term. $\mathbf{y}$ is a distribution over the vocabulary, and $\Pr(w_i | w_{i-n}, \dots, w_{i-1}, w_{i+1}, \dots, w_{i+n}) = y(w_i)$. Eqs. (3.44–3.45) describe a very simple neural network. An advantage is that the resulting model is small and efficient as compared to NLMs. Training the CBOW model is straightforward. We can frame it as finding the maximum likelihood estimation of the parameters of the model. For simplicity, let θ denote the parameters other than $\mathbf{C}$ (i.e., $\theta = \{\mathbf{U}, \mathbf{b}\}$). We have

$$(\hat{\theta}, \hat{\mathbf{C}}) = \arg \max_{\theta, \mathbf{C}} \sum_{i=n+1}^{m-n} \log \Pr_{\theta, \mathbf{C}}(w_i | w_{i-n}, \dots, w_{i-1}, w_{i+1}, \dots, w_{i+n}), \quad (3.46)$$

where m is the length of the word sequence. After training, we can simply drop $\hat{\theta}$ and use $\hat{\mathbf{C}}$ as a word vector look-up table.

- **The continuous skip-gram model (or the skip-gram model).** The skip-gram model is another word prediction model. It models the reverse of the task described in Eqs. (3.44–3.45). To be more precise, our objective is to predict each of the $\pm n$ context words given w_i . This is generally framed as estimating the probability of w_j occurring given w_i ($i-n \leq j \leq i-1$ or $i+1 \leq j \leq i+n$). Figure 3.7 (b) shows the structure of the skip-gram model. The embedding layer deals with w_i as usual. The representation of w_i is given by

$$\mathbf{h} = \mathbf{1}_{w_i} \mathbf{C}. \quad (3.47)$$

It is then passed to a Softmax layer to predict the probability for each context word w_j

(assuming $j = i + k$)¹⁸

$$\mathbf{y}_k = \text{Softmax}(\mathbf{h}\mathbf{V}_k + \mathbf{b}_k), \quad (3.48)$$

where $\mathbf{V}_k$ and $\mathbf{b}_k$ are the parameters of the model ($-n \leq k \leq -1$ and $1 \leq k \leq n$). We have

$$\begin{aligned} \Pr(w_j|w_i) &= \Pr(w_{i+k}|w_i) \\ &= y_k(w_{i+k}), \end{aligned} \quad (3.49)$$

Let θ be a short representation of $\{\mathbf{V}_k\}$ and $\{\mathbf{b}_k\}$. The training problem can be defined as:

$$(\hat{\theta}, \hat{\mathbf{C}}) = \arg\max_{\theta, \mathbf{C}} \sum_{i=n+1}^{m-n} \sum_{\substack{-n \leq k \leq -1, \\ 1 \leq k \leq n}} \log \Pr_{\theta, \mathbf{C}}(w_{i+k}|w_i). \quad (3.50)$$

Both of the above models make an analogy to cloze tests by considering only the pairwise dependency between words. Because they ignore word order and complex syntactic relationships, their predictive distributions are less precise compared to language models. Note, however, that the goal of these models is not to precisely predict missing words given their contexts, but to learn word representations from some task that captures word-word relationships. It is therefore not so important to care about the word prediction performance of the learned model.

Another merit of these models is that they have very simple, easy-to-train architectures. For example, in both models there are no hidden layers and the embedding layer is directly connected to the output layer. These model structures can be seen as instances of log-linear modeling in machine learning: the input variables are linearly transformed into a dense feature vector (e.g., Eq. (3.44)), followed immediately by a log-linear classification function (e.g., Eq. (3.45)).

3.5.2 GloVe

Global vectors, commonly known as **GloVe**, are word embeddings learned by combining the advantages of global matrix factorization and local context window methods [Pennington et al., 2014]. The GloVe method begins by constructing a global word-word co-occurrence matrix (as discussed in Section 3.3) and subsequently formulates a log-bilinear model based on a specific set of theoretical assumptions regarding word meanings.

Given a word-word co-occurrence matrix $\mathbf{M}$, where each cell $M(a, b) = \text{count}(a, b)$ represents the number of times word $a \in V$ co-occurs with word $b \in V$ within a specified

¹⁸When $k > 0$, w_j is a word in the right context window of w_i ; when $k < 0$, w_j is a word in the left context window.

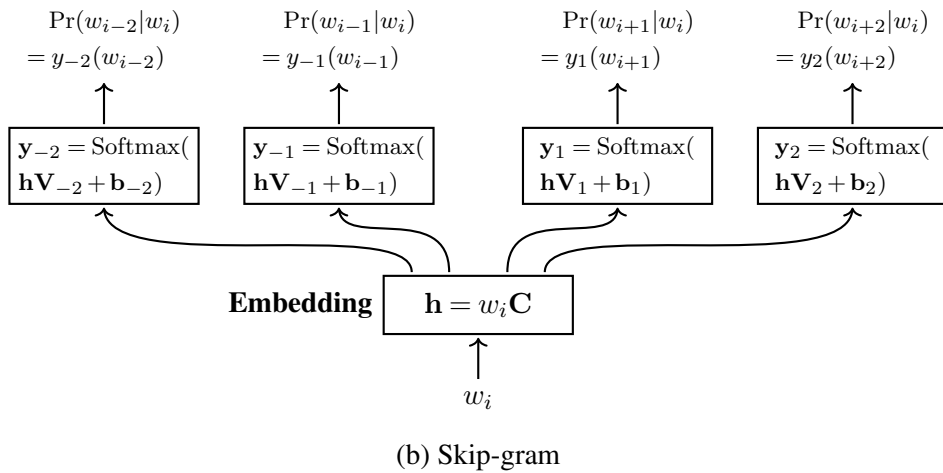
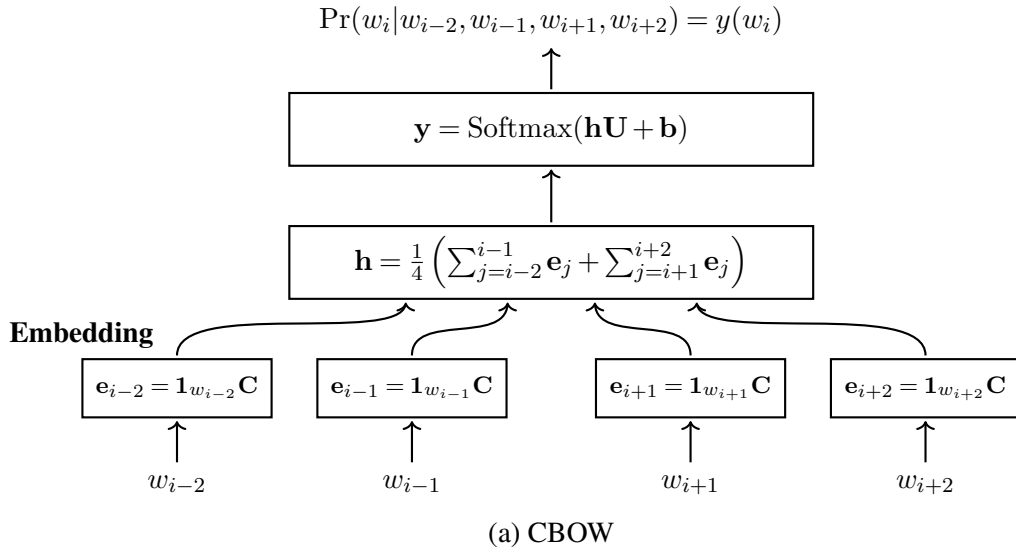


Figure 3.7: The CBOW and skip-gram architectures. The CBOW model computes the probability $\Pr(w_i | w_{i-2}, w_{i-1}, w_{i+1}, w_{i+2})$ where w_i is a word in a sequence and $\{w_{i-2}, w_{i-1}, w_{i+1}, w_{i+2}\}$ are words in the ± 2 context windows. The context representation $\mathbf{h}$ is the mean of the word vectors that are produced through an embedding layer. $\mathbf{h}$ is then fed into a Softmax layer to output a distribution over the vocabulary (i.e., $\mathbf{y}$). The prediction probability of w_i is $\Pr(w_i | w_{i-2}, w_{i-1}, w_{i+1}, w_{i+2}) = y(w_i)$. The skip-gram model is also based on the embedding + softmax structure. It models the probability of each context word w_j given the word w_i . This is achieved by simply computing the output of a standard softmax layer that takes the vector representation of w_i as input. Both the CBOW and skip-gram models are trained in a maximum likelihood fashion. The resulting lookup table of the embedding layer is the word vectors (or embeddings) for the words in the vocabulary.

context window, we can compute the conditional probability $\Pr(b|a)$ as follows:

$$\begin{aligned}\Pr(b|a) &= \frac{\text{count}(a,b)}{\sum_{b'} \text{count}(a,b')} \\ &= \frac{\text{count}(a,b)}{\text{count}(a)},\end{aligned}\tag{3.51}$$

where $\text{count}(a)$ represents the total number of times word a occurs in the corpus.

To understand the motivation behind GloVe, consider an example where we wish to distinguish between the thermodynamic concepts of *air* and *water*. Using Eq. (3.51), we can compute the probability of these target words occurring given various context probe words. The following table displays a small fraction of the $\Pr(b|a)$ matrix derived from a 3.8 billion-word corpus in WMT14:

Entry	$w = \textit{fly}$	$w = \textit{drink}$	$w = \textit{breath}$	$w = \textit{live}$	$w = \textit{flow}$
$\Pr(\textit{air} w)$	1.5×10^{-4}	6.2×10^{-5}	2.2×10^{-4}	1.6×10^{-4}	3.6×10^{-4}
$\Pr(\textit{water} w)$	1.3×10^{-5}	4.1×10^{-4}	1.8×10^{-5}	1.4×10^{-4}	3.0×10^{-4}
$\Pr(\textit{air} w)/\Pr(\textit{water} w)$	11.54	0.15	12.2	1.14	1.2

In this table, $\Pr(\textit{air}|w)$ and $\Pr(\textit{water}|w)$ indicate how strongly *air* and *water* correlate with different probe words w . The crucial insight arises when we examine the probability ratio $\Pr(\textit{air}|w)/\Pr(\textit{water}|w)$, shown in the last row. This ratio effectively isolates the semantic differences between the two target words. When w is highly relevant to *air* but not *water* (e.g., $w = \textit{fly}$ or $w = \textit{breath}$), the ratio is large. Conversely, when w is relevant to *water* but not *air* (e.g., $w = \textit{drink}$), the ratio is small. When w is relevant to both (e.g., $w = \textit{live}$ or $w = \textit{flow}$), or irrelevant to both, the ratio is close to 1.

The foundational premise of GloVe is that word vectors should explicitly encode this ratio of co-occurrence probabilities. A natural starting point is to posit a function F that takes word vectors as input and approximates this ratio:

$$F(\mathbf{e}_a, \mathbf{e}_b, \tilde{\mathbf{e}}_w) = \frac{\Pr(a|w)}{\Pr(b|w)},\tag{3.52}$$

where $\mathbf{e}_a, \mathbf{e}_b \in \mathbb{R}^{d_e}$ are the target vector representations of words a and b , and $\tilde{\mathbf{e}}_w \in \mathbb{R}^{d_e}$ is the context vector representation of the probe word w . Notice the distinction between $\mathbf{e}$ and $\tilde{\mathbf{e}}$: the former is drawn from a target embedding table $\mathbf{C}$, while the latter is drawn from a separate context embedding table $\tilde{\mathbf{C}}$. Utilizing two distinct embedding tables helps mitigate overfitting during training. Ultimately, the final word representation is typically taken as the sum (or average) of the two: $\mathbf{e}_{a,\text{final}} = \mathbf{1}_a(\mathbf{C} + \tilde{\mathbf{C}})$.

Because vector spaces are inherently linear, the most natural way to encode the relationship between the target words a and b is via vector subtraction: $\mathbf{e}_a - \mathbf{e}_b$. Furthermore, to quantify the relationship between this difference and the context word w , we can employ the vector dot

product. This restricts the function F to take a single scalar input:

$$F\left((\mathbf{e}_a - \mathbf{e}_b)\tilde{\mathbf{e}}_w^\top\right) = \frac{\Pr(a|w)}{\Pr(b|w)}. \quad (3.53)$$

To enforce a principled algebraic structure, we require F to be a homomorphism between the additive group in the vector space and the multiplicative group in the probability space. This ensures that vector subtraction maps directly to probability division:

$$F\left((\mathbf{e}_a - \mathbf{e}_b)\tilde{\mathbf{e}}_w^\top\right) = \frac{F(\mathbf{e}_a\tilde{\mathbf{e}}_w^\top)}{F(\mathbf{e}_b\tilde{\mathbf{e}}_w^\top)}. \quad (3.54)$$

The unique continuous solution to Eq. (3.54) is the exponential function, $F(x) = \exp(x)$. Thus, we have:

$$\begin{aligned} F(\mathbf{e}_a\tilde{\mathbf{e}}_w^\top) &= \exp(\mathbf{e}_a\tilde{\mathbf{e}}_w^\top) \\ &= \Pr(a|w) \\ &= \frac{\text{count}(a, w)}{\text{count}(a)}. \end{aligned} \quad (3.55)$$

Taking the logarithm of both sides yields:

$$\mathbf{e}_a\tilde{\mathbf{e}}_w^\top = \log \text{count}(a, w) - \log \text{count}(a). \quad (3.56)$$

An issue with Eq. (3.56) is that the term $\log \text{count}(a)$ breaks the symmetry; exchanging the target word a and context word w (which corresponds to transposing the co-occurrence matrix) should leave the underlying geometry unchanged. To restore this symmetry, the $\log \text{count}(a)$ term is absorbed into two learned bias terms, β_a and $\tilde{\beta}_w$:

$$\mathbf{e}_a\tilde{\mathbf{e}}_w^\top + \beta_a + \tilde{\beta}_w = \log \text{count}(a, w). \quad (3.57)$$

Eq. (3.57) establishes the ideal relationship we want our learned embeddings to satisfy. To enforce this, we formulate a regression problem, aiming to minimize the squared difference between the predicted log co-occurrence and the actual log co-occurrence. The squared loss for a single word pair (a, w) is defined as:

$$\mathcal{L}_{a,w} = \left(\mathbf{e}_a\tilde{\mathbf{e}}_w^\top + \beta_a + \tilde{\beta}_w - \log \text{count}(a, w)\right)^2. \quad (3.58)$$

The total cost function over the entire vocabulary is the weighted sum of these individual losses:

$$L_{\text{GloVe}} = \sum_{a,w \in V} f(\text{count}(a, w)) \cdot \mathcal{L}_{a,w}, \quad (3.59)$$

where $f(\text{count}(a, w))$ is a weighting function designed to prevent rare word pairs from

dominating the loss, while also capping the influence of overly frequent word pairs. [Pennington et al. \[2014\]](#) propose the following weighting function:

$$f(x) = \begin{cases} \left(\frac{x}{x_{\max}}\right)^\alpha & \text{if } x < x_{\max} \\ 1 & \text{otherwise} \end{cases}, \quad (3.60)$$

where $x_{\max}$ and α are hyperparameters (typically, $x_{\max} = 100$ and $\alpha = 3/4$). As such, $f(x)$ penalizes rare word-pairs by assigning them smaller weights, while ensuring that the weight for frequent pairs does not grow unbounded.

Eqs. (3.58–3.59) describe a highly efficient framework for learning word vectors using standard neural network operations (e.g., dot products and scalar additions). A defining characteristic of GloVe is its reliance on a simple log-bilinear objective. Because the training avoids the computationally expensive cross-entropy loss and softmax normalization required by full NLMs and Word2Vec, GloVe is remarkably fast to train. The core intuition is that semantic relationships can be modeled via simple regression on global corpus statistics, rather than relying strictly on probability-based divergence over local context windows.

Another note about the use of global data bears repeating. The co-occurrence matrix encodes information aggregated across the entire corpus. By directly factoring this matrix, GloVe explicitly frames learning as a global optimization problem. While Word2Vec also seeks a globally optimal set of parameters (e.g., maximizing the likelihood over the entire input space), its objective is complex and typically optimized via online learning (stochastic gradient descent on local batches). GloVe, by pre-aggregating the data into a co-occurrence matrix, distills the learning process into a more direct and computationally efficient global model.

3.5.3 Remarks

We have seen in the previous sections how word vectors can be induced using several different methods. We now discuss several important considerations when training or applying these word embeddings in practice.

1. Count-based vs Neural Network-based

The simplicity and interpretability of count-based methods have long been appreciated in the NLP community. Both count-based and neural network-based models fundamentally rely on the distributional hypothesis — the idea that words occurring in similar contexts possess similar meanings. However, count-based methods explicitly construct high-dimensional, sparse matrices based on this assumption, which often leads to data sparsity issues due to the curse of dimensionality.

At the other end of the spectrum are neural network-based models. Rather than relying on explicit co-occurrence counting and manual weighting schemes, these methods treat word vectors as low-dimensional, dense intermediate states within a neural network optimized for a specific background task. By removing explicit linguistic constraints on the specific meaning of each vector dimension, neural network-based models effectively learn latent semantic features that are difficult to manually engineer.

The contrast between these two approaches is similar to the broader comparison of two well-known machine learning paradigms: feature engineering vs. end-to-end representation learning. However, we will not delve deeper into this philosophical debate here. It is simply worth noting that there is no absolute superiority of one specific form over the other. In general, the choice of word vector type depends entirely on the target downstream application and the desired level of interpretability. For example, if the goal is to obtain an explicitly interpretable, mathematically transparent computable form directly from a corpus, factorizing a co-occurrence matrix remains a robust choice. Conversely, if the objective is to extract continuous, dense word vectors intended for seamless integration into a deep neural architecture, neural representation learning is typically preferred.

It should be noted that learning continuous, dense word embeddings has become the dominant paradigm in recent years, demonstrating massive advancements in neural models for NLP. Concurrently, there has been significant academic interest in comparing count-based and neural network-based methods, and revealing the profound theoretical relationships between them [Levy and Goldberg, 2014b; Baroni et al., 2014; Levy and Goldberg, 2014c; Schnabel et al., 2015a; Levy et al., 2015; Gladkova et al., 2016].

2. Shallow Models vs. Deep Models

While neural networks have become the standard approach for inducing word vectors, the model structures introduced thus far in this chapter are simple. Technically, they consist of only one or two neural layers and are therefore categorized as shallow models. A notable example is the **vLBL** word embedding model [Mnih and Kavukcuoglu, 2013], which models the interactions among words using a strictly two-layer neural network. This architecture, which omits even a softmax normalization function, is one of the simplest word embedding models to the best of our knowledge. Despite such simplicity, these models remain highly effective in many scenarios. A primary benefit of shallow models is their computational efficiency and scalability to massive datasets. This efficiency allows them to scale to more complex NLP tasks. A prominent example is the **fastText** system for text classification [Joulin et al., 2017]. It shares a similar architecture with the CBOW model (see Section 3.5.1). In fastText, the input text is represented as a bag of word vectors that are averaged to form a global hidden representation of the text. This is followed immediately by an output layer that maps the hidden representation to a probability distribution over predefined classes. By using this mechanism, the classification objective and the underlying word vectors are trained jointly.

Although shallow models are remarkably effective for learning static word representations, deep neural networks offer significantly greater representational capacity. As with most multi-layer architectures, learning representations via deep neural networks offers several distinct advantages [Telgarsky, 2016]. First, deep models allow us to explore a much larger hypothesis space, potentially discovering superior functional mappings. Second, the injection of multiple non-linear transformations dramatically increases the model's ability to capture highly complex patterns in the data. There are many strategies for leveraging depth in representation learning. The most straightforward approach is to incrementally stack feed-forward, recurrent, or convolutional layers on top of a base embedding layer. With the widespread

adoption of large-scale pretrained models in NLP, the paradigm has shifted toward extracting contextualized representations directly from very deep architectures, achieving state-of-the-art performance across a vast array of NLP benchmarks [Radford et al., 2018; Devlin et al., 2019]. However, this increased modeling power introduces severe computational bottlenecks and training instabilities. Consequently, a substantial body of research focuses on mitigating these optimization challenges [Pascanu et al., 2013; Bapna et al., 2018; Wang et al., 2019; Zhang et al., 2019a; Pham et al., 2019; Li et al., 2020]. In Chapters 4-6, we will explore several successful NLP models built upon these deep neural architectures.

3. Training Objectives

The idea of treating word vectors as model parameters aligns well with latent-variable modeling: a model is parameterized by learnable word vectors, and their optimal values are inferred by optimizing a global objective function. While this learning paradigm is standard, the specific training objective can vary significantly. A primary difficulty is the absence of an explicit objective dedicated solely to supervising word vector representations. A straightforward solution is to leverage well-defined NLP tasks. For instance, we can use word vectors as the input representations for language modeling or text classification systems. As a result, the word vectors serve as standard model parameters and are optimized end-to-end.

An alternative approach is to design novel training objectives. As is typical in machine learning, the design space for such objectives is vast. A common strategy, therefore, is to adapt existing tasks. For example, the training objective of the CBOW model is essentially a generalized word prediction problem and shares a similar mathematical form with traditional language modeling.

We will explore several novel objectives derived from language modeling in Chapter 7. In a strict sense, these training tasks are not inherently about learning word vectors. Rather, they provide a well-defined surrogate objective for representation learning. Furthermore, we typically cannot assume a purely supervised learning scenario: the learned embeddings will ultimately be deployed across diverse downstream systems unknown during the initial training phase. This frames the induction process as a form of self-supervised learning, lacking direct supervision from the ultimate target application. However, when the target application is accessible and labeled task-specific data is available, we can further fine-tune these pre-trained word embeddings to capture domain-specific semantics.

3.6 Evaluating Word Embeddings

Having obtained vector representations of words, we must assess their quality. Ideally, we would evaluate these word vectors against a gold standard. Unfortunately, such a gold standard rarely exists, as it is impractical for human annotators to directly assign high-dimensional, continuous numerical values to capture the semantics of a word. A practical solution is to evaluate the vectors based on the performance of a downstream working system that utilizes them. Broadly, evaluation approaches fall into two categories [Schnabel et al., 2015b]:

- **Extrinsic Evaluation** (or end-to-end testing). We incorporate the word vectors into a downstream NLP system and measure their impact on the system’s overall performance.
- **Intrinsic Evaluation**. We test the ability of the word vectors to capture specific morphological, syntactic, and semantic properties of language.

Below, we briefly describe how these approaches are applied to word vector evaluation.

3.6.1 Extrinsic Evaluation

This approach is widely adopted in practice, as it allows researchers and engineers to quickly examine how a real-world system behaves when its underlying representations are modified. Since many NLP systems take words as input, it is standard practice to replace symbolic word representations with dense word vectors. We have already seen systems of this kind, which typically employ an embedding layer to transform the one-hot representation of each input word into a real-valued vector (e.g., the neural language model in Chapter 2).

Given such a system and a set of pre-trained word vectors, we can use the downstream performance of the system as a proxy measure for the quality of the embeddings. Depending on how the word vectors are utilized, there are two primary training paradigms:

- **Word Vectors as Fixed Parameters**. We freeze the word vectors and only train the remaining parameters of the system as usual.
- **Word Vectors as Initial Parameters** (Fine-Tuning). We initialize the embedding layer with the pre-trained word vectors and update them alongside the rest of the model’s parameters during training.

Both methods leverage the concept of pre-training, and can be extended to many scenarios where part of a model is trained extensively before being applied to a downstream task. Freezing word vectors simplifies the training process and isolates the evaluation of the representations themselves. In contrast, treating the word vectors as learnable parameters increases training complexity but allows the model to fine-tune the representations, adapting them specifically to the target working system.

Note that while extrinsic evaluation is highly practical, its results are heavily dependent on the specific downstream architecture and task. Because developing state-of-the-art NLP systems involves complex architectures and extensive hyperparameter tuning, performance variations cannot always be attributed to the quality of the word embeddings alone. This task dependency is common in NLP. For instance, a tokenization method that excels in a machine translation system might perform poorly in an information retrieval system. Therefore, to rigorously test the generalizability of a given set of word vectors, it is standard practice to evaluate them across a diverse suite of downstream NLP tasks.

3.6.2 Intrinsic Evaluation

Although much of word representation research involves end-to-end tests in NLP applications, it also involves examining the ability of the representation to deal with certain problems, such as interpreting the relationship between two words. There are many ways to design intrinsic

evaluation, each addressing a specific problem. In the following we describe some of these methods. For a more comprehensive survey of intrinsic evaluation methodologies, we refer the reader to the extensive literature on this subject [Baroni et al., 2014; Bakarov, 2018; Rogers et al., 2018].

1. Semantic Relatedness

Modeling the relatedness between words is perhaps the most prevalent method for evaluating the quality of word vectors in NLP [Reisinger and Mooney, 2010; Huang et al., 2012; Baroni et al., 2014]. Fundamentally, this involves computing a spatial distance between words (referred to as **word semantic distance** or simply **word distance**). The motivation is that the geometric distance in the continuous vector space should correlate with human cognitive judgments of semantic relatedness [Rubenstein and Goodenough, 1965]. For example, we expect the vector for *dog* to be geometrically proximal to *wolf*, while *peach* should be distant from *television*. Mathematically, many metrics can be used to calculate the distance or angle between two vectors. A simple and commonly used measure is the Euclidean distance. Alternatively, computing the cosine similarity between two vectors yields a score in the interval $[-1, 1]$ ¹⁹.

In this evaluation setup, we are provided with a dataset of word pairs, each assigned an expected relatedness score by human annotators. Given a pair of words, we correlate this expected human score with the geometric distance measured in the word vector space. The quality of the word vectors is reflected by the strength of this correlation. A primary challenge, however, is the absence of an absolute gold-standard metric for semantic distance. Even human annotators struggle to quantify semantic proximity with absolute numerical precision. A practical alternative is to map the distance into discrete categories or Likert-style rating scales, such as integers in $[1, 5]$ [Reisinger and Mooney, 2010]. This significantly reduces the cognitive load during data annotation. Another evaluation variant requires the model to identify the most similar word to a given target from a small set of candidates (e.g., synonym tests). This avoids predicting absolute values. Instead we only need a mechanism to rank relative similarities [Baroni et al., 2014].

Evaluating relationships between words, however, can be ambiguous due to the fluid nature of language. In general, multiple linguistic factors affect human judgments of relatedness [Faruqui et al., 2016]. For example, *corn* and *cornea* exhibit morphological similarity due to subword overlap, yet they are semantically disjoint. Ambiguity also stems from the very definitions of the tasks. **Semantic relatedness** and **semantic similarity** are often used interchangeably, yet they denote distinct linguistic concepts. For instance, *car* is topically related to *road*, but in a strict taxonomic sense, *car* is similar to *van*.

Furthermore, the meaning of a word is generally context-dependent, and thus confound the evaluation of words with multiple distinct senses (**polysemy**). Broadly speaking, this is an inherent limitation of static word vector models, where every word type is deterministically mapped to a single, fixed vector. To address this, subsequent chapters will introduce contextualized word embeddings, which dynamically generate a distinct computable form for a word

¹⁹In practice, it is common to convert this into a *cosine distance* (defined as $1 - \cos(\theta)$), where a distance of 0 indicates two vectors pointing in the exact same direction, and 1 indicates orthogonal vectors.

based strictly on its surrounding text.

2. Word Analogy

The word analogy task is concerned with modeling analogical relations between pairs of words. This task is motivated by the premise that semantic relationships between words can be captured via simple algebraic operations on their corresponding continuous representations. A canonical example presented by Mikolov et al. [2013d] demonstrates that relational semantics can be described by vector subtraction. Specifically, subtracting the vector for *man* from *king* and adding the vector for *woman* yields a vector remarkably close to that of *queen*:

$$\mathbf{e}_{\text{king}} - \mathbf{e}_{\text{man}} + \mathbf{e}_{\text{woman}} \approx \mathbf{e}_{\text{queen}}. \quad (3.61)$$

Formally, word analogy is the task of comparing two word pairs, (a, a^*) and (b, b^*) . An analogy holds if the semantic relationship between a and a^* mirrors that between b and b^* . This reflects underlying **linguistic regularities** in the vector space, which can be algebraically formulated as:

$$\mathbf{e}_{a^*} - \mathbf{e}_a \approx \mathbf{e}_{b^*} - \mathbf{e}_b. \quad (3.62)$$

Word analogy can thus be framed as an analogical reasoning task: the objective is to predict $\mathbf{e}_{b^*}$ given $\mathbf{e}_a$, $\mathbf{e}_{a^*}$, and $\mathbf{e}_b$. More specifically, we expect the vector $\mathbf{e}_{a^*} - \mathbf{e}_a + \mathbf{e}_b$ to closely approximate $\mathbf{e}_{b^*}$ if (a, a^*) and (b, b^*) share similar linguistic relations. There are many ways to refine this formulation. For example, rather than merely optimizing the spatial distance to predict $\mathbf{e}_{b^*}$, one can evaluate the cosine angle between the difference vectors $\mathbf{e}_{a^*} - \mathbf{e}_a$ and $\mathbf{e}_{b^*} - \mathbf{e}_b$ to capture relational similarity more robustly [Levy and Goldberg, 2014b].

The word analogy task provides a straightforward mechanism to examine the linear algebraic properties of a word vector model. Such characteristics are rarely observed in traditional count-based methods. Remarkably, modern word embedding models exhibit strong linear behavior even though such geometric constraints are not explicitly enforced during training. This emergent property provides researchers with insights into the latent structures learned by these models and their potential downstream applications [Levy and Goldberg, 2014b; Linzen, 2016; Allen and Hospedales, 2019]. However, word analogy accuracy is not a universally indicative metric. In many scenarios, it does not strongly correlate with the performance of downstream NLP systems. As a result, it often serves as an analytical tool for probing the structural and semantic properties of word representations.

3. Word Categorization (or Clustering)

Another method to evaluate whether word vectors align with human semantic understanding is to examine their capacity to be grouped into meaningful categories. This is typically achieved by applying standard clustering algorithms to the word vectors. Ideally, semantically similar words should be assigned to the same cluster, while dissimilar words are separated into distinct clusters. For instance, *apple*, *grape*, *peach*, and *orange* should naturally cluster together under the broader semantic category of fruits. An advantage of this evaluation approach is that many

clustering algorithms and established word categorization benchmarks are readily applicable. However, as with most unsupervised clustering tasks, practical challenges must be addressed, such as predetermining the optimal number of clusters.

Because most clustering methods in machine learning rely on computing spatial distances between data points, word clustering is rooted in the concept of semantic relatedness. However, unlike the tasks discussed in Section 3.6.2, clustering evaluates the global structure rather than explicitly judging individual pairwise distance scores. While this highlights the intrinsic connections among different evaluation protocols, it also means that word clustering has the inherent limitations of these related methods. Most notably, it is difficult to establish an absolute gold-standard criterion for clustering quality, as semantic taxonomies are subjective, and words can be categorized in multiple ways depending on the chosen semantic perspective.

4. Subconscious Evaluation

The goal of subconscious (or cognitive) evaluation is to examine the correlation between the use of word vectors and human subconscious behaviors or brain activity during reading. A wide variety of psycholinguistic phenomena can serve as evaluation proxies [Mitchell and Lapata, 2010]. A well-known paradigm is **priming**, which investigates how a person responds to sequential stimuli [Schacter and Buckner, 1998; Tulving and Schacter, 1990; Wiggs and Martin, 1998]. For example, an experiment can be designed to measure the speed at which a participant reads a given word (referred to as the **target word**) immediately following another word (the **prime word**) [Meyer and Schvaneveldt, 1971; Lund, 1995; McNamara, 2005]. If a target word t is processed more rapidly following word a compared to word b , it indicates a stronger cognitive association between t and a . This psychological latency can then serve as a proxy to evaluate the semantic similarity between their corresponding word vectors. To measure reading latency, a popular methodology is the **self-paced reading** task²⁰. An alternative is to use eye-tracking devices to automatically record precise information regarding eye movements and fixations. By using these techniques, many datasets and methodologies have been developed to study a variety of psycholinguistic issues [Mitchell and Lapata, 2010; Hutchison et al., 2013; Lapesa and Evert, 2014; Klerke et al., 2015; Søgaaard, 2016; Auguste et al., 2017].

In addition to tracking behavioral responses during reading, we can directly monitor brain activity using neuroimaging techniques, such as functional magnetic resonance imaging (fMRI) and electroencephalography (EEG) [Devereux et al., 2010; Søgaaard, 2016; Bhattasali et al., 2020]. For example, language comprehension elicits specific neural activation patterns in the brain. Therefore, we can attempt to correlate these physiological activation patterns directly with the computable form of word meaning provided by vector representations. However, a limitation is that our understanding of the underlying cognitive mechanisms driving these processes remains incomplete, making it difficult to sufficiently correlate the results of such psycholinguistic studies with the performance of real-world NLP systems [Baroni et al., 2014; Bakarov, 2018].

²⁰In self-paced reading, the text is segmented into words or phrases, and the participant presses a button to advance the display segment by segment.

5. Linguistically Motivated Evaluation

Linguistically motivated evaluation is based on the assumption that data-driven word vectors should reflect the structural and semantic knowledge encoded in linguistic resources. An interesting approach to performing such evaluation is to align the word vectors with representations of dictionary entries [Tsvetkov et al., 2015; Acs and Kornai, 2016]. The quality of the word embeddings is then quantified by the correlation between these continuous vectors and the discrete linguistic representations²¹. Beyond standard dictionaries, word vectors can be evaluated against topological structures within semantic networks, such as WordNet. In this way, evaluation metrics can be enriched by employing graph-based algorithms to measure the structural alignment between the vector space and these semantic resources [Agirre et al., 2009].

3.6.3 Visualization

By treating word vectors as multi-dimensional data points, we can adopt dimensionality reduction techniques to project them onto a two- or three-dimensional space for visualization. Such projections enable us to visually analyze the semantic patterns encoded within these vectors and explore the latent relationships between words. One method is PCA which seeks a linear mapping from a high-dimensional space to a low-dimensional space (see Section 3.3.3). Another well-known method is **t-distributed stochastic neighbor embedding (t-SNE)** [Hinton and Roweis, 2002; Van der Maaten and Hinton, 2008]. It is a non-linear dimensionality reduction method, and has been widely used in visualizing high-dimensional data. Apart from these, one can consider the methods presented in Section 3.3.3 as well as those tailored for visualizing word vectors [Zhang et al., 2019b; Liu et al., 2017].

3.7 Summary

In this chapter we discussed two interesting problems in NLP: tokenization and word (or token) representation. First, we introduced models for dividing a sentence into units that are meaningful and/or well suited for downstream tasks. Second, we introduced the idea of word vector models with particular attention to learning both count-based high-dimensional models and real-valued low-dimensional models. While most of these models are simple, they are often used in complex NLP systems and form the basis of many advanced models, as will be shown in the following chapters.

Tokenization (or segmentation) is an important process in NLP, commonly as a preprocessing step for many applications [Webster and Kit, 1992]. However, the use of the term *tokenization* is slightly misleading as it originally refers to the basic computer science process of dividing a string into substrings. In modern NLP, tokenization draws on concepts from multiple sub-fields. Linguistically, it is deeply tied to two core questions: how words are morphologically composed and how they syntactically form sentences. It is therefore natural to use morphological and syntactic theories to define the basic units of a language, leading to many

²¹In this context, a linguistic representation is typically a handcrafted feature vector derived from a formal linguistic resource, such as a lexicon.

rule-based tokenization systems covering a variety of languages. From a machine learning perspective, tokenization has long been cast as a problem of learning token boundaries from data in either a supervised or unsupervised manner [Mielke et al., 2021]. A common approach is to first annotate some tokenized text with human knowledge about what basic language units should be, and then learn to tokenize on this annotated data (see Section 3.1.3). More recently, learning tokenizers without linguistic constraints has been found to be promising (see Section 3.1.4). Since natural languages are themselves sets of characters or byte sequences, it is also possible to segment a sentence into characters or bytes [Ling et al., 2015; Lee et al., 2017]. Tokenization-free methods in general help when one wants a language-independent tokenizer and a simpler pipeline for processing the text.

From a mathematical standpoint, tokenization can be thought of as a mapping from the input data to a sequence of variables. Thus, the concept of tokenization can be generalized by relaxing the assumption that both the input and output variables are constrained to discrete values. Recent image and speech processing systems, for example, transform continuous sensory data (such as pixels and acoustic signals) into sequences of vector-based “tokens” [Schneider et al., 2019; Dosovitskiy et al., 2021]. Some interesting extensions of these ideas are even to transform image and speech data to a sequence of discrete indices, thereby aligning computer vision and speech processing more closely with traditional NLP paradigms [Oord et al., 2017; Baevski et al., 2020; Hsu et al., 2021].

Having divided the input text into smaller segments, the natural next step is to represent these pieces in some way that captures their underlying features. While representing language units as dense numerical vectors has become the de facto standard in modern NLP, the work on vector representation dates back to the very early days of computational linguistics. According to many popular textbooks and papers [Manning and Schütze, 1999; Jurafsky and Martin, 2008], the idea of using a statistical distribution to represent word meaning, known as **distributional semantics**, emerged in the 1950s alongside the rise of empiricism. During that era, much of the research was heavily influenced by Harris’s distributionalism [Harris, 1954] and related philosophical frameworks [Firth, 1957; Wittgenstein, 1953]. Concurrently, Osgood [1952] proposed defining the meaning of a concept psychologically, as a point within a multidimensional space. These ideas significantly shaped how linguists and NLP researchers conceptualized word meaning over the following decades.

Modern approaches to distributional semantics appeared in the 1990s, mainly as a result of the revival of empiricism in artificial intelligence [Church, 2011]. Most of these were driven by the distributional hypothesis: words having similar meanings are more likely to occur in similar contexts. In response, a number of methods were developed, differing in the way the contexts are modeled. For example, a context can be the words in a context-window, or the words with a relation to the given word in a syntax tree. Apart from those mentioned in Section 3.3, methods that are not covered in this chapter include hyperspace analogue of language (HAL) [Lund and Burgess, 1996], distributional memory [Baroni and Lenci, 2010], dependency-based semantic space models [Padó and Lapata, 2007], and so on. For comprehensive surveys of distributional semantics, the reader can refer to dedicated literature [Lenci, 2018; Mitchell and Lapata, 2010]. Note that most of the above-mentioned work can be thought of as instances of the

vector space model which can deal with problems beyond lexical semantics. In compositional distributional semantics, for example, the meaning of a phrase or sentence is derived by performing systematic algebraic operations on its constituent word vectors [Clark et al., 2008; Mitchell and Lapata, 2010; Blacoe and Lapata, 2012].

While distributional models have attracted attention in the NLP community for many years, word embedding models that learn low-dimensional, real-valued word vectors directly from texts have been a predominant approach recently. As described in Sections 3.4- 3.5, rather than explicitly constructing sparse co-occurrence matrices, these models implicitly leverage the distributional hypothesis by optimizing a vector of latent attributes to predict surrounding contexts. The resulting model is an extension of the feature-based semantic model [Markman, 2013]. A recognized difference with traditional feature-based methods is that we do not need to manually define the features. We instead take these features as parameters of the model, and train them in an end-to-end fashion.

Formulating word representation as an end-to-end learning problem brings with it several benefits. A primary advantage is that new features can be found because no constraints are placed on how these features are learned and interpreted. Furthermore, as demonstrated in Section 3.6.2, word vectors obtained in this way indeed show some linguistic properties, though the word embedding models are not trained to achieve this. Another benefit is that the word embedding models also fall within the broader vector space model paradigm, enabling the easy use of word vectors in various applications. There are also many examples of methods that attempt to improve standard word embedding systems. Researchers have successfully infused pre-trained embeddings with external linguistic knowledge [Levy and Goldberg, 2014a; Cotterell and Schütze, 2015; Tissier et al., 2017] and aligned vector spaces to induce universal representations in multiple languages [Klementiev et al., 2012; Mikolov et al., 2013b; Ammar et al., 2020; Smith et al., 2017; Artetxe et al., 2017].

Widely associated with neural models in NLP, the idea of distributed representation has been successfully generalized to structural units far beyond individual words. This includes learning dense representations for sentences [Le and Mikolov, 2014; Kalchbrenner et al., 2014; Kiros et al., 2015; Hill et al., 2016; Arora et al., 2017; Lin et al., 2017; Conneau et al., 2017] as well as complex tree and graph structures [Socher et al., 2011; Perozzi et al., 2014; Tai et al., 2015; Grover and Leskovec, 2016]. In particular, contextualized representations of words, though not discussed in this chapter, are generally appreciated for modeling sequential data [McCann et al., 2017; Peters et al., 2018; Devlin et al., 2019].

Bibliography

- [Acs and Kornai, 2016] Judit Acs and András Kornai. Evaluating embeddings on dictionary-based similarity. In *Proceedings of the 1st Workshop on Evaluating Vector-Space Representations for NLP*, pages 78–82, 2016.
- [Agirre et al., 2009] Eneko Agirre, Enrique Alfonseca, Keith Hall, Jana Kravalová, Marius Pasca, and Aitor Soroa. A study on similarity and relatedness using distributional and wordnet-based approaches. In *Proceedings of Human Language Technologies: The 2009 Annual Conference of the North American Chapter of the Association for Computational Linguistics*, pages 19–27, 2009.
- [Allen and Hospedales, 2019] Carl Allen and Timothy Hospedales. Analogies explained: Towards understanding word embeddings. In *International Conference on Machine Learning*, pages 223–231. PMLR, 2019.
- [Ammar et al., 2020] Waleed Ammar, George Mulcaire, Yulia Tsvetkov, Guillaume Lample, Chris Dyer, and Noah A Smith. Massively multilingual word embeddings. In *Proceedings of the 8th International Conference on Learning Representations (ICLR)*, 2020.
- [Aronoff and Fudeman, 2011] Mark Aronoff and Kirsten Fudeman. *What is morphology?*, volume 8. John Wiley & Sons, 2011.
- [Arora et al., 2017] Sanjeev Arora, Yingyu Liang, and Tengyu Ma. A simple but tough-to-beat baseline for sentence embeddings. In *International conference on learning representations*, 2017.
- [Artetxe et al., 2017] Mikel Artetxe, Gorka Labaka, and Eneko Agirre. Learning bilingual word embeddings with (almost) no bilingual data. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 451–462, 2017.
- [Auguste et al., 2017] Jeremy Auguste, Arnaud Rey, and Benoit Favre. Evaluation of word embeddings against cognitive processes: primed reaction times in lexical decision and naming tasks. In *Proceedings of the 2nd workshop on evaluating vector space representations for NLP*, pages 21–26, 2017.
- [Baevski et al., 2020] Alexei Baevski, Steffen Schneider, and Michael Auli. vq-wav2vec: Self-supervised learning of discrete speech representations. In *Proceedings of ICLR 2020*, 2020.
- [Bakarov, 2018] Amir Bakarov. A survey of word embeddings evaluation methods. *arXiv preprint arXiv:1801.09536*, 2018.
- [Bapna et al., 2018] Ankur Bapna, Mia Xu Chen, Orhan Firat, Yuan Cao, and Yonghui Wu. Training deeper neural machine translation models with transparent attention. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3028–3033, 2018.
- [Barber, 2012] David Barber. *Bayesian Reasoning and Machine Learning*. Cambridge University Press, 2012.

- [Baroni and Lenci, 2010] Marco Baroni and Alessandro Lenci. Distributional memory: A general framework for corpus-based semantics. *Computational Linguistics*, 36(4):673–721, 2010.
- [Baroni et al., 2014] Marco Baroni, Georgiana Dinu, and Germán Kruszewski. Don’t count, predict! a systematic comparison of context-counting vs. context-predicting semantic vectors. In *Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 238–247, 2014.
- [Bengio et al., 2003] Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Jauvin. A neural probabilistic language model. *Journal of Machine Learning Research*, 3:1137–1155, 2003.
- [Bhattachali et al., 2020] Shohini Bhattachali, Jonathan Brennan, Wen-Ming Luh, Berta Franzluebbers, and John Hale. The alice datasets: fMRI & EEG observations of natural language comprehension. In *Proceedings of the 12th Language Resources and Evaluation Conference*, pages 120–125, 2020.
- [Blacoe and Lapata, 2012] William Blacoe and Mirella Lapata. A comparison of vector-based representations for semantic composition. In *Proceedings of the 2012 joint conference on empirical methods in natural language processing and computational natural language learning*, pages 546–556, 2012.
- [Blei, 2012] David M Blei. Probabilistic topic models. *Communications of the ACM*, 55(4):77–84, 2012.
- [Blei et al., 2003] David M Blei, Andrew Y Ng, and Michael I Jordan. Latent dirichlet allocation. *Journal of machine Learning research*, 3(Jan):993–1022, 2003.
- [Brown et al., 1993] Peter F. Brown, Stephen A. Della Pietra, Vincent J. Della Pietra, and Robert L. Mercer. The mathematics of statistical machine translation: Parameter estimation. *Computational Linguistics*, 19(2):263–311, 1993.
- [Buttcher et al., 2016] Stefan Buttcher, Charles LA Clarke, and Gordon V Cormack. *Information retrieval: Implementing and evaluating search engines*. MIT Press, 2016.
- [Church, 2011] Kenneth Church. A pendulum swung too far. *Linguistic Issues in Language Technology*, 6, 2011.
- [Church and Hanks, 1990] Kenneth Ward Church and Patrick Hanks. Word association norms, mutual information, and lexicography. *Computational Linguistics*, 16(1):22–29, 1990. URL <https://aclanthology.org/J90-1003>.
- [Clark et al., 2008] Stephen Clark, Bob Coecke, and Mehrnoosh Sadrzadeh. A compositional distributional model of meaning. In *Proceedings of the Second Quantum Interaction Symposium (QI-2008)*, pages 133–140. Oxford, 2008.
- [Cline and Dhillon, 2014] Alan Kaylor Cline and Inderjit S. Dhillon. Computation of the singular value decomposition. In Leslie Hogben, editor, *Handbook of Linear Algebra (2dn ed.)*. CRC Press, 2014.
- [Conneau et al., 2017] Alexis Conneau, Douwe Kiela, Holger Schwenk, Loïc Barrault, and Antoine Bordes. Supervised learning of universal sentence representations from natural language inference data. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 670–680, 2017.
- [Cotterell and Schütze, 2015] Ryan Cotterell and Hinrich Schütze. Morphological word-embeddings. In *Proceedings of the 2015 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1287–1292, 2015.
- [Deerwester et al., 1990] Scott Deerwester, Susan T Dumais, George W Furnas, Thomas K Landauer,

- and Richard Harshman. Indexing by latent semantic analysis. *Journal of the American society for information science*, 41(6):391–407, 1990.
- [Dempster et al., 1977] Arthur P Dempster, Nan M Laird, and Donald B Rubin. Maximum likelihood from incomplete data via the em algorithm. *Journal of the Royal Statistical Society: Series B (Methodological)*, 39(1):1–22, 1977.
- [Devereux et al., 2010] Barry Devereux, Colin Kelly, and Anna Korhonen. Using fmri activation to conceptual stimuli to evaluate methods for extracting conceptual representations from corpora. In *Proceedings of the NAACL HLT 2010 First Workshop on Computational Neurolinguistics*, pages 70–78, 2010.
- [Devlin et al., 2019] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, 2019.
- [Dosovitskiy et al., 2021] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *Proceedings of ICLR 2021*, 2021.
- [Faruqui et al., 2016] Manaal Faruqui, Yulia Tsvetkov, Pushpendre Rastogi, and Chris Dyer. Problems with evaluation of word embeddings using word similarity tasks. In *Proceedings of the 1st Workshop on Evaluating Vector-Space Representations for NLP*, pages 30–35, 2016.
- [Firth, 1957] John R Firth. A synopsis of linguistic theory, 1930-1955. *Studies in linguistic analysis*, 1957.
- [Freedman et al., 2007] David Freedman, Robert Pisani, and Roger Purves. *Statistics (4th ed.)*. W. W. Norton & Company, 2007.
- [Friedl, 2006] Jeffrey Friedl. *Mastering Regular Expressions (3rd ed.)*. O’Reilly Media, 2006.
- [Gage, 1994] Philip Gage. A new algorithm for data compression. *C Users Journal*, 12(2):23–38, 1994.
- [Gladkova et al., 2016] Anna Gladkova, Aleksandr Drozd, and Satoshi Matsuoka. Analogy-based detection of morphological and semantic relations with word embeddings: what works and what doesn’t. In *Proceedings of the NAACL Student Research Workshop*, pages 8–15, 2016.
- [Grover and Leskovec, 2016] Aditya Grover and Jure Leskovec. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 855–864, 2016.
- [Guyon and Elisseeff, 2003] Isabelle Guyon and André Elisseeff. An introduction to variable and feature selection. *Journal of machine learning research*, 3(Mar):1157–1182, 2003.
- [Harris, 1954] Zellig S Harris. Distributional structure. *Word*, 10(2-3):146–162, 1954.
- [Hill et al., 2016] Felix Hill, Kyunghyun Cho, and Anna Korhonen. Learning distributed representations of sentences from unlabelled data. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1367–1377, 2016.
- [Hinton and Roweis, 2002] Geoffrey E Hinton and Sam Roweis. Stochastic neighbor embedding. *Advances in neural information processing systems*, 15, 2002.

- [Hsu et al., 2021] Wei-Ning Hsu, Benjamin Bolte, Yao-Hung Hubert Tsai, Kushal Lakhotia, Ruslan Salakhutdinov, and Abdelrahman Mohamed. Hubert: Self-supervised speech representation learning by masked prediction of hidden units. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 29:3451–3460, 2021.
- [Huang et al., 2012] Eric H Huang, Richard Socher, Christopher D Manning, and Andrew Y Ng. Improving word representations via global context and multiple word prototypes. In *Proceedings of the 50th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 873–882, 2012.
- [Hutchison et al., 2013] Keith A Hutchison, David A Balota, James H Neely, Michael J Cortese, Emily R Cohen-Shikora, Chi-Shing Tse, Melvin J Yap, Jesse J Bengson, Dale Niemeyer, and Erin Buchanan. The semantic priming project. *Behavior research methods*, 45(4):1099–1114, 2013.
- [Jackendoff, 1992] Ray S Jackendoff. *Semantic structures*, volume 18. MIT press, 1992.
- [Jolliffe, 2002] Ian T Jolliffe. *Principal component analysis for special types of data*. Springer, 2002.
- [Joulin et al., 2017] Armand Joulin, Édouard Grave, Piotr Bojanowski, and Tomáš Mikolov. Bag of tricks for efficient text classification. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers*, pages 427–431, 2017.
- [Jurafsky and Martin, 2008] Dan Jurafsky and James H. Martin. *Speech and Language Processing (2nd ed.)*. Prentice Hall, 2008.
- [Kalchbrenner et al., 2014] Nal Kalchbrenner, Edward Grefenstette, and Phil Blunsom. A convolutional neural network for modelling sentences. In *Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 655–665, 2014.
- [Kelly and Stone, 1975] Edward F. Kelly and Philip J. Stone. *Computer recognition of English word senses*. American Elsevier Pub, 1975.
- [Kiros et al., 2015] Ryan Kiros, Yukun Zhu, Russ R Salakhutdinov, Richard Zemel, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. Skip-thought vectors. *Advances in neural information processing systems*, 28, 2015.
- [Klementiev et al., 2012] Alexandre Klementiev, Ivan Titov, and Binod Bhattarai. Inducing crosslingual distributed representations of words. In *Proceedings of COLING 2012*, pages 1459–1474, 2012.
- [Klerke et al., 2015] Sigrid Klerke, Héctor Martínez Alonso, and Anders Søgaard. Looking hard: Eye tracking for detecting grammaticality of automatically compressed sentences. In *Proceedings of the 20th Nordic Conference of Computational Linguistics (NODALIDA 2015)*, pages 97–105, 2015.
- [Knight, 2018] Linda Knight. The sparrow tweets, 2018.
- [Koehn et al., 2007] Philipp Koehn, Hieu Hoang, Alexandra Birch, Chris Callison-Burch, Marcello Federico, Nicola Bertoldi, Brooke Cowan, Wade Shen, Christine Moran, Richard Zens, Chris Dyer, Ondřej Bojar, Alexandra Constantin, and Evan Herbst. Moses: Open source toolkit for statistical machine translation. In *Proceedings of the 45th Annual Meeting of the Association for Computational Linguistics Companion Volume Proceedings of the Demo and Poster Sessions*, pages 177–180, 2007.
- [Kudo, 2018] Taku Kudo. Subword regularization: Improving neural network translation models with multiple subword candidates. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 66–75, 2018.
- [Kudo and Richardson, 2018] Taku Kudo and John Richardson. Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the*

- 2018 *Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 66–71, 2018.
- [Kupiec, 1992] Julian Kupiec. Robust part-of-speech tagging using a hidden markov model. *Computer Speech & Language*, 6:225–242, 1992.
- [Lafferty et al., 2001] John Lafferty, Andrew McCallum, and Fernando Pereira. Conditional random fields: Probabilistic models for segmenting and labeling sequence data. In *Proceedings of the 18th International Conference on Machine Learning 2001*, pages 282–289, 2001.
- [Landauer et al., 1998] Thomas K Landauer, Peter W Foltz, and Darrell Laham. An introduction to latent semantic analysis. *Discourse processes*, 25(2-3):259–284, 1998.
- [Lapesa and Evert, 2014] Gabriella Lapesa and Stefan Evert. A large scale evaluation of distributional semantic models: Parameters, interactions and model selection. *Transactions of the Association for Computational Linguistics*, 2:531–546, 2014.
- [Lawson, 2003] Mark V. Lawson. *Finite Automata (1st ed.)*. Chapman and Hall/CRC, 2003.
- [Le and Mikolov, 2014] Quoc Le and Tomas Mikolov. Distributed representations of sentences and documents. In *International conference on machine learning*, pages 1188–1196. PMLR, 2014.
- [Lee et al., 2017] Jason Lee, Kyunghyun Cho, and Thomas Hofmann. Fully character-level neural machine translation without explicit segmentation. *Transactions of the Association for Computational Linguistics*, 5:365–378, 2017.
- [Lenci, 2018] Alessandro Lenci. Distributional models of word meaning. *Annual review of Linguistics*, 4:151–171, 2018.
- [Levy and Goldberg, 2014] Omer Levy and Yoav Goldberg. Dependency-based word embeddings. In *Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 302–308, 2014a.
- [Levy and Goldberg, 2014] Omer Levy and Yoav Goldberg. Linguistic regularities in sparse and explicit word representations. In *Proceedings of the eighteenth conference on computational natural language learning*, pages 171–180, 2014b.
- [Levy and Goldberg, 2014] Omer Levy and Yoav Goldberg. Neural word embedding as implicit matrix factorization. *Advances in neural information processing systems*, 27, 2014c.
- [Levy et al., 2015] Omer Levy, Yoav Goldberg, and Ido Dagan. Improving distributional similarity with lessons learned from word embeddings. *Transactions of the association for computational linguistics*, 3:211–225, 2015.
- [Li et al., 2020] Bei Li, Ziyang Wang, Hui Liu, Yufan Jiang, Quan Du, Tong Xiao, Huizhen Wang, and Jingbo Zhu. Shallow-to-deep training for neural machine translation. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 995–1005, 2020.
- [Lin et al., 2017] Zhouhan Lin, Minwei Feng, Cicero Nogueira dos Santos, Mo Yu, Bing Xiang, Bowen Zhou, and Yoshua Bengio. A structured self-attentive sentence embedding. In *Proceedings of the 5th International Conference on Learning Representations (ICLR)*, 2017.
- [Ling et al., 2015] Wang Ling, Chris Dyer, Alan W Black, Isabel Trancoso, Ramón Fernandez, Silvio Amir, Luis Marujo, and Tiago Luís. Finding function in form: Compositional character models for open vocabulary word representation. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pages 1520–1530, 2015.

- [Linzen, 2016] Tal Linzen. Issues in evaluating semantic spaces using word analogies. In *Proceedings of the 1st Workshop on Evaluating Vector-Space Representations for NLP*, pages 13–18, 2016.
- [Liu and Motoda, 2012] Huan Liu and Hiroshi Motoda. *Feature selection for knowledge discovery and data mining*, volume 454. Springer Science & Business Media, 2012.
- [Liu et al., 2017] Shusen Liu, Peer-Timo Bremer, Jayaraman J Thiagarajan, Vivek Srikumar, Bei Wang, Yarden Livnat, and Valerio Pascucci. Visual exploration of semantic relationships in neural word embeddings. *IEEE transactions on visualization and computer graphics*, 24(1):553–562, 2017.
- [Lund, 1995] Kevin Lund. Semantic and associative priming in high-dimensional semantic space. In *Proc. of the 17th Annual conferences of the Cognitive Science Society, 1995*, 1995.
- [Lund and Burgess, 1996] Kevin Lund and Curt Burgess. Producing high-dimensional semantic spaces from lexical co-occurrence. *Behavior research methods, instruments, & computers*, 28(2):203–208, 1996.
- [Manning and Schütze, 1999] Chris Manning and Hinrich Schütze. *Foundations of Statistical Natural Language Processing*. The MIT Press, 1999.
- [Manning et al., 2008] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [Markman, 2013] Arthur B Markman. *Knowledge representation*. Psychology Press, 2013.
- [McCallum et al., 2000] Andrew McCallum, Dayne Freitag, and Fernando C. N. Pereira. Maximum entropy markov models for information extraction and segmentation. In *Proceedings of the Seventeenth International Conference on Machine Learning*, pages 591–598, 2000.
- [McCann et al., 2017] Bryan McCann, James Bradbury, Caiming Xiong, and Richard Socher. Learned in translation: Contextualized word vectors. *Advances in neural information processing systems*, 30, 2017.
- [McClave and Sincich, 2006] James T. McClave and Terry Sincich. *Statistics (10th ed.)*. Prentice Hall, 2006.
- [McNamara, 2005] Timothy P McNamara. *Semantic priming: Perspectives from memory and word recognition*. Psychology Press, 2005.
- [Meyer and Schvaneveldt, 1971] David E Meyer and Roger W Schvaneveldt. Facilitation in recognizing pairs of words: evidence of a dependence between retrieval operations. *Journal of experimental psychology*, 90(2):227, 1971.
- [Mielke et al., 2021] Sabrina J. Mielke, Zaid Alyafeai, Elizabeth Salesky, Colin Raffel, Manan Dey, Matthias Gallé, Arun Raja, Chenglei Si, Wilson Y. Lee, Benoît Sagot, and Samson Tan. Between words and characters: A brief history of open-vocabulary modeling and tokenization in nlp. *arXiv preprint arXiv:2112.10508*, 2021.
- [Mikolov et al., 2013] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. In *Proceedings of the International Conference on Learning Representations (ICLR 2013)*, 2013a.
- [Mikolov et al., 2013] Tomas Mikolov, Quoc V Le, and Ilya Sutskever. Exploiting similarities among languages for machine translation. *arXiv preprint arXiv:1309.4168*, 2013b.
- [Mikolov et al., 2013] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeffrey Dean. Distributed representations of words and phrases and their compositionality. In *Proceedings of*

- the 26th International Conference on Neural Information Processing Systems - Volume 2*, pages 3111–3119, 2013c.
- [Mikolov et al., 2013] Tomas Mikolov, Wen-tau Yih, and Geoffrey Zweig. Linguistic regularities in continuous space word representations. In *Proceedings of the 2013 conference of the north american chapter of the association for computational linguistics: Human language technologies*, pages 746–751, 2013d.
- [Mitchell and Lapata, 2010] Jeff Mitchell and Mirella Lapata. Composition in distributional models of semantics. *Cognitive science*, 34(8):1388–1429, 2010.
- [Mnih and Kavukcuoglu, 2013] Andriy Mnih and Koray Kavukcuoglu. Learning word embeddings efficiently with noise-contrastive estimation. *Advances in neural information processing systems*, 26, 2013.
- [Montague, 1974] Richard Montague. Universal grammar. In R. Thomason, editor, *Formal Philosophy: Selected Papers of Richard Montague*. Yale University Press, 1974.
- [Oord et al., 2017] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. *Advances in neural information processing systems*, 30, 2017.
- [Osgood, 1952] Charles E Osgood. The nature and measurement of meaning. *Psychological bulletin*, 49(3):197, 1952.
- [Padó and Lapata, 2007] Sebastian Padó and Mirella Lapata. Dependency-based construction of semantic space models. *Computational Linguistics*, 33(2):161–199, 2007.
- [Pascanu et al., 2013] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. In *International conference on machine learning*, pages 1310–1318. PMLR, 2013.
- [Pennington et al., 2014] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. Glove: Global vectors for word representation. In *Proceedings of Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, 2014.
- [Perozzi et al., 2014] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. Deepwalk: Online learning of social representations. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 701–710, 2014.
- [Peters et al., 2018] Matthew E. Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. Deep contextualized word representations. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, 2018.
- [Pham et al., 2019] Ngoc-Quan Pham, Thai-Son Nguyen, Jan Niehues, Markus Müller, Sebastian Stüker, and Alexander Waibel. Very deep self-attention networks for end-to-end speech recognition. *arXiv preprint arXiv:1904.13377*, 2019.
- [Porter, 1980] Martin F Porter. An algorithm for suffix stripping. *Program*, 1980.
- [Radford et al., 2018] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. *OpenAI*, 2018.
- [Reisinger and Mooney, 2010] Joseph Reisinger and Raymond Mooney. Multi-prototype vector-space models of word meaning. In *Human Language Technologies: The 2010 Annual Conference of the North American Chapter of the Association for Computational Linguistics*, pages 109–117, 2010.

- [Rogers et al., 2018] Anna Rogers, Shashwath Hosur Ananthakrishna, and Anna Rumshisky. What’s in your embedding, and how it predicts task performance. In *Proceedings of the 27th International Conference on Computational Linguistics*, pages 2690–2703, 2018.
- [Rubenstein and Goodenough, 1965] Herbert Rubenstein and John B Goodenough. Contextual correlates of synonymy. *Communications of the ACM*, 8(10):627–633, 1965.
- [Schacter and Buckner, 1998] Daniel L Schacter and Randy L Buckner. Priming and the brain. *Neuron*, 20(2):185–195, 1998.
- [Schnabel et al., 2015] Tobias Schnabel, Igor Labutov, David Mimno, and Thorsten Joachims. Evaluation methods for unsupervised word embeddings. In *Proceedings of the 2015 conference on empirical methods in natural language processing*, pages 298–307, 2015a.
- [Schnabel et al., 2015] Tobias Schnabel, Igor Labutov, David Mimno, and Thorsten Joachims. Evaluation methods for unsupervised word embeddings. In *Proceedings of the 2015 conference on empirical methods in natural language processing*, pages 298–307, 2015b.
- [Schneider et al., 2019] Steffen Schneider, Alexei Baevski, Ronan Collobert, and Michael Auli. wav2vec: Unsupervised pre-training for speech recognition. In *INTERSPEECH*, 2019.
- [Schuster and Nakajima, 2012] Mike Schuster and Kaisuke Nakajima. Japanese and korean voice search. In *Proceedings of International Conference on Acoustics, Speech and Signal Processing*, pages 5149–5152, 2012.
- [Sennrich et al., 2016] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, 2016.
- [Smith et al., 2017] Samuel L Smith, David HP Turban, Steven Hamblin, and Nils Y Hammerla. Offline bilingual word vectors, orthogonal transformations and the inverted softmax. In *Proceedings of the 5th International Conference on Learning Representations (ICLR)*, 2017.
- [Socher et al., 2011] Richard Socher, Cliff C Lin, Chris Manning, and Andrew Y Ng. Parsing natural scenes and natural language with recursive neural networks. In *Proceedings of the 28th international conference on machine learning (ICML-11)*, pages 129–136, 2011.
- [Søgaard, 2016] Anders Søgaard. Evaluating word embeddings with fmri and eye-tracking. In *Proceedings of the 1st workshop on evaluating vector-space representations for NLP*, pages 116–121, 2016.
- [Solorio-Fernández et al., 2020] Saúl Solorio-Fernández, J Ariel Carrasco-Ochoa, and José Fco Martínez-Trinidad. A review of unsupervised feature selection methods. *Artificial Intelligence Review*, 53(2):907–948, 2020.
- [Stewart, 1993] Gilbert W Stewart. On the early history of the singular value decomposition. *SIAM review*, 35(4):551–566, 1993.
- [Szabó, 2020] Zoltán Gendler Szabó. Compositionality. In Edward N. Zalta, editor, *The Stanford Encyclopedia of Philosophy*. Metaphysics Research Lab, Stanford University, Fall 2020 edition, 2020.
- [Tai et al., 2015] Kai Sheng Tai, Richard Socher, and Christopher D Manning. Improved semantic representations from tree-structured long short-term memory networks. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 1556–1566, 2015.

- [Telgarsky, 2016] Matus Telgarsky. Benefits of depth in neural networks. In *Conference on learning theory*, pages 1517–1539. PMLR, 2016.
- [Tissier et al., 2017] Julien Tissier, Christophe Gravier, and Amaury Habrard. Dict2vec: Learning word embeddings using lexical dictionaries. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 254–263, 2017.
- [Tsvetkov et al., 2015] Yulia Tsvetkov, Manaal Faruqui, Wang Ling, Guillaume Lample, and Chris Dyer. Evaluation of word vector representations by subspace alignment. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pages 2049–2054, 2015.
- [Tulving and Schacter, 1990] Endel Tulving and Daniel L Schacter. Priming and human memory systems. *Science*, 247(4940):301–306, 1990.
- [Van der Maaten and Hinton, 2008] Laurens Van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of machine learning research*, 9(11), 2008.
- [Wang et al., 2019] Qiang Wang, Bei Li, Tong Xiao, Jingbo Zhu, Changliang Li, Derek F Wong, and Lidia S Chao. Learning deep transformer models for machine translation. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 1810–1822, 2019.
- [Webster and Kit, 1992] Jonathan J Webster and Chunyu Kit. Tokenization as the initial phase in nlp. In *Proceedings of COLING 1992 volume 4: The 14th international conference on computational linguistics*, 1992.
- [Wiggs and Martin, 1998] Cheri L Wiggs and Alex Martin. Properties and mechanisms of perceptual priming. *Current opinion in neurobiology*, 8(2):227–233, 1998.
- [Wittgenstein, 1953] Ludwig Wittgenstein. *Philosophical investigations. Philosophische Untersuchungen*. Macmillan, 1953.
- [Wright and Ma, 2022] John Wright and Yi Ma. *High-Dimensional Data Analysis with Low-Dimensional Models: Principles, Computation, and Applications*. Cambridge University Press, 2022.
- [Zhang et al., 2019] Biao Zhang, Ivan Titov, and Rico Sennrich. Improving deep transformer with depth-scaled initialization and merged attention. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 898–909, 2019a.
- [Zhang et al., 2019] Juexiao Zhang, Yubei Chen, Brian Cheung, and Bruno A Olshausen. Word embedding visualization via dictionary learning. *arXiv preprint arXiv:1910.03833*, 2019b.
- [Zhao et al., 2006] Hai Zhao, Chang-Ning Huang, Mu Li, and Bao-Liang Lu. Effective tag set selection in Chinese word segmentation via conditional random field modeling. In *Proceedings of the 20th Pacific Asia Conference on Language, Information and Computation*, pages 87–94, 2006.

